

Geração automática de exercícios de programação e código. Explicações usando grandes modelos de linguagem

Sami Sarsa
Universidade
Aalto Espoo,
Finlândia sami.sarsa@aalto.fi

Arto Hellas
Universidade Aalto
Espoo, Finlândia
arto.hellas@aalto.fi

Paulo Denny
Universidade de Auckland
Auckland, Nova Zelândia
paul@cs.auckland.ac.nz

Juho Leinonen
Universidade Aalto
Espoo, Finlândia
juho.2.leinonen@aalto.fi

RESUMO

Este artigo explora as capacidades de geração de linguagem natural de grandes modelos de linguagem, com aplicação na produção de dois tipos de recursos de aprendizagem comuns em cursos de programação. Utilizando o OpenAI Codex como modelo de linguagem de grande escala, criamos exercícios de programação (incluindo soluções de exemplo e casos de teste) e explicações de código, avaliando-os qualitativa e quantitativamente. Nossos resultados sugerem que a maior parte do conteúdo gerado automaticamente é original e coerente, e em alguns casos, pronto para uso imediato. Ao criar exercícios, constatamos que é surpreendentemente fácil influenciar tanto os conceitos de programação quanto os temas contextuais neles contidos, simplesmente fornecendo palavras-chave como entrada para o modelo. Nossa análise indica que há um valor significativo em modelos generativos massivos de aprendizado de máquina como ferramenta para instrutores, embora ainda seja necessário algum controle para garantir a qualidade do conteúdo gerado antes de ser disponibilizado aos alunos. Discutimos ainda as implicações do OpenAI Codex e de ferramentas semelhantes para o ensino introdutório de programação e destacamos futuras linhas de pesquisa que têm o potencial de melhorar a qualidade da experiência educacional tanto para professores quanto para alunos.

CONCEITOS CCS

• **Temas sociais e profissionais** ÿ Educação em informática; •
Metodologias de computação ÿ Geração de linguagem natural.

PALAVRAS-CHAVE

Geração de linguagem natural, OpenAI Codex, GPT-3, CS1, Exercícios de programação, Explicações de código, Robosourcing, Geração de exercícios, Geração de recursos, Feedback automatizado, Grandes modelos de linguagem

Formato de referência da ACM:

Sami Sarsa, Paul Denny, Arto Hellas e Juho Leinonen. 2022. Geração automática de exercícios de programação e explicações de código usando grandes modelos de linguagem. In *Anais da Conferência Internacional de Pesquisa em Educação Computacional da ACM de 2022* (ICER 2022), 7 a 11 de agosto de 2022, Lugano e evento virtual, Suíça. ACM, Nova York, NY, EUA, 17 páginas. <https://doi.org/10.1145/3501385.3543957>

1 INTRODUÇÃO

A criação de um curso introdutório de programação envolve o design de materiais didáticos, o desenvolvimento de tarefas, a criação de oportunidades de feedback e o planejamento do fluxo do curso [25]. Proporcionar oportunidades de aprendizagem ativa – que envolve tanto a prática quanto a reflexão – é especialmente importante para a aprendizagem de programação [72]. Uma abordagem específica que ganhou popularidade devido à ampla disponibilidade de ferramentas de correção automática que podem gerar feedback imediato é o uso de muitos exercícios curtos de programação que os alunos usam para desenvolver domínio por meio da prática regular [4, 21, 85]. Essas abordagens se tornaram tão populares que Finnie-Ansley et al. sugerem que elas podem formar uma pedagogia característica na educação em computação [27, 76].

O desenvolvimento de exercícios de programação adequados envolve múltiplas facetas, incluindo a criação de enunciados de problemas, soluções de código de exemplo e testes automatizados. Criar uma quantidade suficiente de exercícios inéditos para formar um recurso útil é um desafio significativo para os educadores, e particularmente difícil em uma era onde as soluções para exercícios existentes são rapidamente publicadas e compartilhadas online. De fato, escrever boas perguntas e bons testes para verificá-las é um desafio fundamental [53, 91]. Assim, não é surpreendente que pesquisadores tenham explorado abordagens colaborativas para a geração de recursos, como o crowdsourcing, mas estas não estão isentas de problemas, pois sofrem com limitações relacionadas à motivação dos alunos e à qualidade do conteúdo [15, 65, 71].

Consideremos um possível enunciado de problema para um exercício de programação que poderia aparecer no início de um curso típico de CS2 focado em programação orientada a objetos [67]. Neste exemplo, a linguagem é Python e o objetivo é familiarizar os alunos com os conceitos básicos de atributos, métodos e chamada de métodos, bem como praticar a manipulação de dicionários internos.

É concedida permissão para fazer cópias digitais ou impressas, totais ou parciais, desta obra para uso pessoal ou em sala de aula, sem custo, desde que as cópias não sejam feitas ou distribuídas para fins lucrativos ou comerciais e que as cópias contenham este aviso e a citação completa na primeira página. Os direitos autorais dos componentes desta obra que pertencem a terceiros que não o (s) autor(es) devem ser respeitados. É permitido fazer resumos com a devida atribuição. Para copiar, republicar, postar em servidores ou redistribuir para listas, é necessária permissão prévia específica e/ou o pagamento de uma taxa. Solicite permissões pelo e-mail permissions@acm.org. ICER 2022, 7 a 11 de agosto de 2022, Lugano e evento virtual, Suíça © 2022. Direitos autorais reservados ao(s) proprietário(s)/autor(es). Direitos de publicação licenciados à ACM. ACM ISBN 978-1-4503-9194-8/22/08...\$15.00 <https://doi.org/10.1145/3501385.3543957>

Escreva uma classe chamada Fis her man thatis inicializado

y com ,adictionaryoffish : 1 0
y , " Truta " : 2 0 , "Atum" : 5 } . A classe
y deve ter um método chamado catch y recebe um parâmetro y , qual
retorna o número de peixes capturados. , um nome de peixe e ,

y A classe também possui um método chamado
y jogar fora, y e diminui o que leva um nome de peixe
número de peixes em
y o dicionário por 1

Por exemplo, o coque de:

```
fi she rman = Fi s he rman ( { " Salmão " : 1 0 , " Truta " : 2 0 ,  
y "Atum " : 5 } )  
fi she rm an . pegar ( " Salmão " )  
fi she rm an . throw_away ( " T rout " )  
fi she rm an . pegar ( " Atum " )  
imprimir (fi ela rm um . peixe )
```

mostrar impressão:

```
{ " Salmão " : 1 1 , " Truta " : 1 9 , "Atum": 6}
```

Um aluno que opta por trabalhar neste exercício de programação.
pode produzir uma solução semelhante à seguinte.

```
classe Fis her man ( ) :  
    def init _ , _ ( selffish = peixe , peixe ) :  
        auto .  
  
    def pegar ( self , peixe ) :  
        se peixe em autopeixe :  
            auto . peixe [ peixe ] += 1  
        outro :  
            auto . peixe [ peixe ] = 1  
  
    def throw_away ( selffish ) :  
        se peixe em autopescas :  
            auto . peixe [ peixe ] y= 1  
        se selffish [ peixe ] == 0 :  
            do próprio eu . peixe [ peixe ]
```

Assim que o aluno enviar sua solução, ele normalmente
receber algum tipo de feedback do avaliador automático. Se houver um
Se encontrar um erro no código, o aluno pode tentar percorrer a solução linha por
linha para ajudá-lo a localizar o problema. Se ficar preso,
Eles podem entrar em contato com um assistente de programa para obter ajuda, caso em que
O assistente pode pedir ao aluno que explique seu código e, em conjunto,
Acompanhe-os passo a passo. Esse tipo de comunicação , semelhante à
técnica de "depuração com patinho de borracha", pode ser uma maneira eficaz.
de explorar a própria compreensão [64]. Se o aluno precisar de
De grande ajuda, o assistente de ensino pode até mesmo ajudar a gerar um
explicação para eles. Uma explicação passo a passo do programa acima.
pode ser o seguinte.

- (1) Criamos um dicionário chamado peixe onde armazenaremos nossos peixes e sua quantidade.
- (2) Criamos uma classe chamada Pescador.
- (3) Criamos um construtor para a classe Fisherman onde nós Inicialize o dicionário de peixes.
- (4) Criamos um método chamado captura que recebe um peixe como um parâmetro.

- (5) Se o peixe já estiver no dicionário de peixes, incrementamos seu valor por 1.
- (6) Se o peixe não estiver no dicionário de peixes, nós o adicionamos e definimos seu valor para 1.
- (7) Criamos um método chamado throw_away que recebe um peixe como parâmetro.
- (8) Se o peixe estiver no dicionário de peixes, decrementamos seu valor por 1.
- (9) Se o valor do peixe chegar a 0, excluimos o peixe do dicionário.

Gerar esse tipo de explicação e ajudar os alunos a decifram seu próprio código em conjunto pode levar um tempo considerável.
de tempo e esforço para os professores. Esse esforço cresce proporcionalmente com o tamanho da coorte, assim como o desafio de desenvolver repositórios de exercícios práticos que se adaptam às variadas necessidades de aprendizagem de turmas maiores e grupos mais diversos. Isso torna a ideia de gerar automaticamente esse tipo de material uma perspectiva empolgante. Como uma questão de
Na verdade, o exercício de programação acima, sua solução e a explicação do código foram todos gerados automaticamente pelo OpenAI Codex, que

É um modelo generativo de PNL (Processamento de Linguagem Natural) para criar código e textos relacionados a código.
Além do exercício, da solução e da explicação do código, a OpenAI A Codex também gerou um conjunto de casos de teste para o exercício que poderiam ser usado para verificar automaticamente as soluções tentadas. Discutimos o geração de conjuntos de testes mais adiante no artigo. Dadas as conhecidas dificuldades em manter a integridade dos bancos de questões existentes [3], É importante notar que a descrição do problema em si é, tanto quanto podemos...
contar, algo totalmente novo. No momento em que escrevo, a descrição do problema Não retorna nenhuma correspondência relevante (usando qualquer combinação de frases). fragmentos da descrição) em mecanismos de busca como o Google ou em sites como Chegg ou StackOverflow, que são frequentemente usados. pelos alunos para encontrar soluções a partir de enunciados de problemas.

Um trabalho muito recente de Finnie-Ansley et al. explorou as implicações do OpenAI Codex no ensino de programação, mas a partir do perspectiva de avaliação da precisão do código-fonte gerado por o modelo para resolver questões típicas de testes e exames CS1 [27]. Isto Estudos anteriores focaram-se principalmente nos desafios que a tecnologia impõe aos educadores, incluindo sérias questões de integridade acadêmica. dependência excessiva por parte dos novatos e confusão causada pela geração de código incorreto ou código com estilo ruim. Neste trabalho, exploramos as oportunidades proporcionadas por essa nova tecnologia. Em vez disso. de focar na geração de código-fonte, que é o mais

Com base na funcionalidade divulgada do OpenAI Codex, investigamos principalmente a fundo as funcionalidades do OpenAI Codex. a geração de artefatos de linguagem natural – tanto programação exercícios e explicações de código – que podem agregar valor a ambos instrutores e alunos.

- RQ1 Em que medida os exercícios de programação são criados usando O Codex da OpenAI é sensato, inovador e facilmente aplicável?
- RQ2 Quão abrangentes e precisas são as explicações em linguagem natural das soluções de código introdutórias do OpenAI Codex? exercícios de programação?

Nosso trabalho oferece informações sobre a utilidade do OpenAI Codex como Este artigo aborda uma parte do conjunto de ferramentas de um professor de um curso introdutório de programação e discute o potencial adicional dessas ferramentas. Neste trabalho, focamos na aplicabilidade do OpenAI Codex para o geração de exercícios de programação e para criação de feedback

Geração automática de exercícios de programação e explicações de código usando grandes modelos de linguagem. ICER 2022, 7 a 11 de agosto de 2022, Lugano e evento virtual, Suíça.

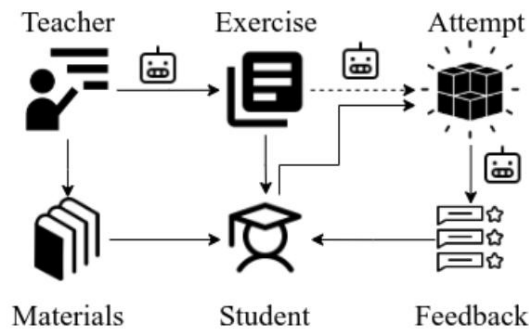


Figura 1: Ciclo de vida de um exercício de programação. O professor cria exercícios de programação e materiais de aprendizagem. Os alunos estudam os materiais e os exercícios e criam tentativas de resolução dos exercícios. Os alunos recebem feedback sobre suas tentativas. Neste trabalho, indicado pelas setas sólidas com um robô, exploramos o uso do OpenAI Codex para a criação de exercícios de programação e para fornecer feedback sobre as tentativas dos alunos nesses exercícios. A seta tracejada com um robô representa o trabalho anterior de Finnie-Ansley et al. [27], que explorou a capacidade do Codex em resolver exercícios introdutórios de programação.

desde as tentativas dos alunos até os exercícios de programação. A Figura 1, que ilustra um ciclo de vida simplificado para um exercício de programação, fornece algum contexto para nossas contribuições.

2. CONTEXTO

2.1 Prática e Feedback em Cursos Introdutórios de Programação

Cursos introdutórios de programação em todo o mundo intercalam teoria e prática, proporcionando aos alunos oportunidades de aprender a escrever programas guiados por enunciados de exercícios, sistemas automatizados de avaliação e professores. Os cursos e as tarefas são geralmente elaborados de forma a serem progressivamente mais complexos, introduzindo novos conceitos gradualmente e buscando evitar sobrecarregar os alunos, ou seja, evitando a sobrecarga cognitiva [20]. Tal estrutura pode ser vista como um andaime que apoia os alunos em sua zona de desenvolvimento proximal [87], isto é, sua área de habilidades e conhecimentos onde eles ainda não conseguem ter sucesso sozinhos, mas onde podem ter sucesso com orientação. À medida que o aluno aprende, sua zona de desenvolvimento proximal também se modifica.

O design de cursos de programação e de tarefas curriculares frequentemente reflete a ideia de prática deliberada [23], que é um tipo de prática sistemática e proposital focada na melhoria do desempenho em uma tarefa específica. A prática deliberada contínua é sustentada por perseverança [19], ou seja, paixão e dedicação a objetivos de longo prazo, mesmo quando a busca desses objetivos parece difícil. A motivação para as tarefas é influenciada pelo design das mesmas; enquanto tarefas muito fáceis têm uma alta expectativa de sucesso, seu valor de utilidade é baixo e, de acordo com a teoria da expectativa-valor [69], os alunos podem ter pouca motivação para realizá-las. Por outro lado, tarefas muito difíceis também têm pouca utilidade e podem levar à baixa motivação [69], contribuindo provavelmente de forma negativa para

sentimentos de autoeficácia [6]. Uma prática no ensino de cursos introdutórios de programação que busca evitar que os alunos se deparem prematuramente com tarefas muito complexas é o uso de muitos exercícios de programação pequenos que ajudam a desenvolver domínio por meio da prática regular [4, 21, 85]. Como os alunos têm diferentes históricos, diferentes habilidades e zonas de desenvolvimento proximal em diferentes estágios de evolução, cada aluno provavelmente se beneficiaria de um conjunto personalizado de tarefas, talvez até com dicas contextuais ajustadas aos seus próprios interesses. Este último ponto é apoiado por trabalhos anteriores em educação computacional, sugerindo que a familiaridade dos alunos com o contexto de um problema pode ser potencialmente útil [44]. No entanto, desenvolver bancos de exercícios tão extensos seria muito trabalhoso [53, 91].

Além de criar tarefas de programação, os professores frequentemente elaboram oportunidades de feedback para as tarefas do curso. Uma abordagem comum para fornecer feedback em cursos introdutórios de programação é o uso de sistemas automatizados de avaliação [2, 35, 60], que, no mínimo, fornecem feedback sobre a correção das tarefas de programação submetidas para avaliação. Como o feedback desempenha um papel considerável na aprendizagem [32], além de influenciar as abordagens de aprendizagem simplesmente por ser oferecido [86], ele deve ser dado com cuidado; o feedback pode tanto melhorar quanto diminuir a autoeficácia [32]. Em geral, o feedback formativo – feedback dado como parte do processo de aprendizagem – é preferível ao feedback somativo, ou seja, feedback dado após o processo de aprendizagem [40, 77].

Em particular, o feedback formativo pode ser usado para auxiliar a aprendizagem autorregulada e a metacognição, ajudando os alunos a se tornarem melhores aprendizes [77].

As práticas em sala de aula e a forma como o ensino de programação é organizado também são importantes [84]. Em particular, oportunidades de feedback podem ser incluídas nas sessões de sala de aula e laboratório. Por exemplo, tanto a instrução entre pares [13] quanto a programação em pares [90] criam oportunidades para reflexão. Na instrução entre pares, a reflexão é parcialmente guiada pelo professor responsável pelas questões da instrução entre pares, enquanto na programação em pares, os alunos interagem e refletem sobre o programa em que estão trabalhando. Os alunos tendem a gostar de programação em pares [1] e também aprendem a raciocinar e explicar o código.

2.2 Explicações de Código e sua Avaliação

A capacidade de raciocinar sobre o código e explicar seu propósito é uma habilidade fundamental que os iniciantes desenvolvem à medida que adquirem experiência [56]. A relação entre a capacidade de um aluno de explicar o código e habilidades relacionadas, como rastreamento de código e escrita de código, tem sido o foco de muitas pesquisas anteriores em educação em computação [51, 52, 75, 83]. As evidências desse conjunto de trabalhos geralmente sugerem que a competência em explicar o código se desenvolve após as habilidades de rastreamento de código de nível inferior e antes das habilidades de escrita de código de nível superior. Essa hierarquia forma a base de uma teoria de ensino de programação recentemente proposta por Xie et al., na qual os alunos primeiro desenvolvem a capacidade de explicar o propósito de modelos de código reutilizáveis antes de escrever código para resolver novos

A avaliação das habilidades de rastreamento e escrita de código geralmente é simples, pois as respostas são objetivas e, portanto, podem ser facilmente automatizadas. As questões de rastreamento de código normalmente pedem aos alunos que determinem a saída ou o valor armazenado em uma variável após a execução de um bloco de código fornecido e são frequentemente apresentadas em forma de pergunta de resposta [50]. Hassan e Zilles propõem um novo método de 'rastreamento reverso'

formato de questão que não é trivial mesmo quando os alunos têm acesso a um computador [31], e Lehtinen et al. exploraram a criação automática de questões de múltipla escolha a partir do próprio código dos alunos [43]. Diversas abordagens e ferramentas para ajudar os alunos a desenvolver habilidades de rastreamento de código também foram relatadas [68, 93]. Questões de escrita de código exigem que os alunos produzam código a partir da descrição de um problema. Muitas ferramentas para correção automática de questões de escrita de código surgiram à medida que as instituições se afastam dos exames em papel [57, 81] e buscam fornecer feedback imediato sobre tarefas e trabalhos de programação [7, 45, 82].

A avaliação da capacidade de explicar código é menos direta, pois as explicações geralmente são dadas em linguagem natural e podem ser fornecidas em diferentes níveis de abstração. Um método popular para avaliar a capacidade de explicar código é o formato de pergunta "explique em linguagem simples", em que os alunos são solicitados a explicar a finalidade de um bloco de código fornecido. Esse tipo de pergunta foi estudado pela primeira vez como parte do projeto BRACElet [88], no qual as respostas dos alunos foram classificadas pelos pesquisadores de acordo com os quatro primeiros níveis da taxonomia SOLO [8]. O nível mais alto, relacional, caracterizou respostas que descreviam, em um alto nível de abstração, como o código se comportaria em todas as entradas possíveis. Um exemplo clássico do trabalho BRACElet é a resposta "ele verifica se o array está ordenado" para descrever um código que compara elementos adjacentes em um array dentro de um loop [49]. O próximo nível mais alto, multiestrutural, foi usado para classificar respostas que forneciam uma descrição linha por linha do código, mas não conseguiam declarar sucintamente sua finalidade. Pesquisas subsequentes estabeleceram uma forte correlação entre habilidades de escrita de código e a capacidade de construir respostas para perguntas de explicação em linguagem simples no nível relacional [56]. Mais recentemente, foram exploradas abordagens para avaliar questões de explicação em linguagem simples em exames, incluindo uma rubrica validada para orientar a avaliação manual [10] e uma ferramenta automatizada que apresentou precisão semelhante à de assistentes de ensino treinados [29]. Ambas as abordagens, como a classificação SOLO comumente usada em pesquisas, diferenciam entre respostas em um nível abstrato e aquelas que são descrições linha por linha do código.

Claramente, a capacidade de explicar a finalidade de um código correto em um nível abstrato é uma habilidade importante para iniciantes desenvolverem. No entanto, quando o código está incorreto, a situação é mais complexa. Para códigos que contêm erros, tentativas de descrever sua finalidade no nível relacional são prematuras e provavelmente não identificarão os erros. De fato, Perkins et al. argumentam que a capacidade de ler o que um trecho de código realmente faz, em vez do que pensamos que ele possa fazer em uma primeira inspeção rápida, é uma importante habilidade de depuração [63]. Eles descrevem a análise linha por linha do código como "rastreamento minucioso", o que reflete, em certa medida, as "simulações mentais" que Soloway argumenta que devem ser ensinadas explicitamente aos alunos [79]. Portanto, é possível que explicações multiestruturais do código, em nível linha por linha, possam trazer algum benefício para fins de depuração. Dado que a análise passo a passo de código pode ser mentalmente exigente, receber uma explicação do próprio código pode reduzir as demandas cognitivas associadas à depuração [55], e evidências de outros domínios educacionais sugerem que receber explicações produzidas por outros pode melhorar o aprendizado [89]. No ensino de computação, técnicas como programação em pares [30] e feedback entre pares baseado em concepções errôneas [39] oferecem algumas oportunidades para analisar o código com outras pessoas, mas nem sempre são eficazes.

viável, mas pode não ser adequado para avaliações individuais de alunos. Portanto, a geração automática de explicações de código, especialmente para auxiliar os alunos na depuração, é uma ideia atraente e que se torna viável com a introdução de ferramentas como o OpenAI Codex.

2.3 Modelos de Aprendizado de Máquina para Geração de Código

Recentemente, houve um grande progresso em modelos generativos de linguagem natural, como o GPT-3 da OpenAI [9], que são capazes de gerar texto que pode ser difícil de distinguir do texto escrito por humanos [9]. Esses são modelos de aprendizado profundo e seu desempenho depende tanto de um vasto número de parâmetros para os modelos (175 bilhões no caso do GPT-3) quanto de um extenso corpus de texto para treinamento (570 GB de texto para o GPT-3). O Codex [11], também da OpenAI, é um modelo GPT semelhante ao GPT-3, mas foi ajustado usando código disponível publicamente no GitHub com o objetivo de traduzir linguagem natural em código-fonte e vice-versa, e de gerar ou completar automaticamente código-fonte a partir de código-fonte como entrada.

Além do Codex da OpenAI, outros modelos generativos de aprendizado de máquina capazes de gerar linguagem natural a partir de código-fonte e/ou vice-versa foram desenvolvidos. Um dos primeiros modelos desse tipo é o CodeBERT da Microsoft [26]. O CodeBERT foi treinado com pares de linguagem natural e linguagem de programação e é capaz de gerar documentação de código-fonte automaticamente, de forma semelhante ao Codex. Outro modelo apresentado recentemente é o AlphaCode da DeepMind [48], que é capaz de ter um desempenho equivalente ao de um concorrente mediano quando confrontado com problemas em nível de competição de programação.

Esses modelos recentes têm múltiplas aplicações. Uma que foi proposta é ajudar os programadores a corrigir código inseguro. Pearce et al. [61] analisaram o desempenho do Codex e de modelos semelhantes para reparar código-fonte contendo falhas de segurança e descobriram que, ao fornecer um prompt cuidadosamente construído para o modelo, eles conseguiram corrigir problemas de segurança em programas em alguns casos. Outro estudo de Pearce et al. [62] analisou a possibilidade de utilizar o Codex para engenharia reversa. Em seu estudo, eles forneceram ao Codex código descompilado e solicitaram que o Codex explicasse a finalidade do código. Seus resultados indicaram que existe algum potencial na utilização de modelos como o Codex para engenharia reversa, já que pouco mais da metade das perguntas feitas pelos autores foram respondidas corretamente. No entanto, eles sugerem que é necessário trabalho contínuo e propõem o ajuste fino do modelo, fornecendo-lhe dados de treinamento específicos ao contexto (no caso deles, código-fonte descompilado).

Finalmente, relevante para o presente artigo, trabalhos muito recentes no domínio da matemática demonstraram que grandes modelos de linguagem podem resolver e gerar novos problemas com sucesso [17]. Curiosamente, para resolver os problemas matemáticos, os autores utilizam o Codex para gerar soluções baseadas em código a partir de instruções em linguagem natural, que são então executadas para realizar os cálculos.

2.4 Potencial do Codex no Ensino de Computação O caso de uso mais comum do Codex é a geração de novo código a partir de um fragmento de código fornecido ou de uma descrição em linguagem natural de um problema. O GitHub Copilot, um plug-in para editor de texto que utiliza a tecnologia do OpenAI Codex, promete gerar código.

sugestões “para linhas inteiras ou funções inteiras diretamente no seu editor”. De fato, o slogan do Copilot é: “seu programador de IA em dupla”. Um desenvolvedor que utiliza este plug-in normalmente recebe sugestões de código em tempo real enquanto digita, ou fornece explicitamente uma descrição em linguagem natural (por exemplo, como um comentário no código) e, em seguida, recebe sugestões mais abrangentes, como funções inteiras, quase que imediatamente. Em muitos casos, são fornecidas várias sugestões, que o desenvolvedor pode simplesmente percorrer e aceitar ou rejeitar.

Este caso de uso de geração de código poderia ser aplicado produtivamente de diversas maneiras em contextos de ensino de computação. Por exemplo, como soluções modelo foram propostas como um mecanismo de apoio na programação introdutória [58, 59], os alunos poderiam gerar soluções modelo com o Codex para problemas históricos de tarefas, testes e exames, para os quais soluções podem não existir de outra forma. Eles também poderiam gerar soluções alternativas corretas para um problema que já resolveram, para refletir sobre sua própria solução e comparar diferentes algoritmos e construções de linguagem. À medida que a precisão do Codex melhora com o tempo, a pedagogia introdutória de computação pode se afastar da codificação de baixo nível e se aproximar da decomposição e resolução de problemas.

No entanto, o Codex não se limita a tarefas de geração de código, podendo gerar saída em linguagem natural a partir de entradas em código ou em prosa. Neste artigo, exploramos como essa capacidade pode ser usada para dar suporte a dois novos casos de uso relacionados ao ciclo de vida de exercícios de programação (ver Figura 1). O primeiro deles diz respeito à geração de exercícios de programação pelo instrutor. Dado um exercício existente como entrada, exploramos se o Codex pode gerar novas variações do exercício que poderiam então ser aplicadas aos alunos. O segundo caso de uso diz respeito à geração de feedback para os alunos. Dado o código-fonte como entrada, exploramos se o Codex pode produzir feedback útil em linguagem natural sobre esse código, particularmente em termos de ajudar os alunos a detectar erros antes do envio para avaliação. Em geral, o uso de uma ferramenta como o Codex para gerar problemas práticos para estudantes de computação em vários formatos e para fornecer feedback útil aos alunos sobre seu progresso nesses problemas parece oferecer um grande potencial.

No contexto do ensino de programação, Finnie-Ansley et al. [27] estudaram o potencial do Codex para resolver tarefas introdutórias de programação. Eles descobriram que o Codex era capaz de responder corretamente à maioria dos problemas introdutórios de programação e que, quando submetido a questões típicas de provas, seu desempenho era superior ao do aluno médio. Os autores observam que, considerando o desempenho do Codex e, principalmente, o rápido progresso nessa área, há consequências claras para os cursos introdutórios de programação. Por exemplo, à medida que modelos como o Codex, capazes de apresentar bom desempenho em tarefas de programação, se tornam cada vez mais comuns, torna-se cada vez mais fácil para os alunos usarem esses modelos para escrever código, essencialmente cometendo um novo tipo de plágio, o que pode exigir a utilização de detecção de plágio baseada em processos [33, 46, 54]. Enquanto Finnie-Ansley et al. se concentraram principalmente nos potenciais desafios que modelos como o Codex podem introduzir nas salas de aula de programação introdutória, nosso foco neste artigo é explorar as oportunidades potenciais que esses modelos podem proporcionar no ensino de programação.

3 METODOLOGIA

3.1 Utilizando o Codex

Semelhante aos modelos GPT-3 da OpenAI, o Codex pode ser utilizado tanto programaticamente por meio de uma API quanto por meio de uma interface web. O usuário fornece um estímulo, ou seja, uma solicitação, ao Codex como entrada, e o Codex gera novo conteúdo como saída com base no estímulo fornecido. Por exemplo, dada uma descrição em linguagem natural do comportamento desejado, o Codex frequentemente gera o código-fonte de um programa que fornece essa funcionalidade.

Para gerar conteúdo, é possível especificar uma “sequência de parada” personalizada, que interrompe a geração de texto ao criar tal sequência. Outras opções relevantes que utilizamos neste estudo incluem a contagem máxima de tokens, que controla o comprimento do conteúdo gerado, e a “temperatura”, que controla a “criatividade” ou “aleatoriedade” do modelo. Um valor de temperatura mais baixo reduz ainda mais as chances de o modelo gerar tokens menos prováveis, diminuindo a aleatoriedade no processo de criação. Com qualquer valor de temperatura, no entanto, o modelo não é determinístico e podem ocorrer diferenças no conteúdo criado entre as execuções, embora isso seja mais comum com valores de temperatura mais altos.

Como o estímulo inicial fornecido ao Codex direciona o modelo para o conteúdo que deve ser gerado, além de gerar código, podemos, por exemplo, fornecer ao Codex um exercício de programação existente e algumas palavras relacionadas ao contexto. Isso guia o Codex a tentar criar conteúdo semelhante ao estímulo inicial, que, neste caso, seria um exercício similar, mas com um contexto específico. Para referência, considerando, por exemplo, o modelo de linguagem natural GPT-3 (no qual o Codex se baseia), usar um estímulo inicial sobre cachorros provavelmente levará a resultados relacionados a cachorros.

3.2 Criação de Exercícios de Programação e Explicações de Código

3.2.1 Seleção de entradas

para o Codex. Para fins das análises neste artigo, selecionamos um pequeno conjunto de exercícios que têm sido utilizados em pesquisas sobre educação em computação e que são frequentemente usados nos contextos de ensino dos pesquisadores, que utilizam a abordagem de muitos exercícios pequenos [4]. Concentramo-nos em quatro exercícios de programação: 1) uma variante do problema de velocidade [83], onde os alunos trabalham com condicionais e retorno de valores; 2) uma variante do FizzBuzz [5], onde os alunos trabalham com condicionais e listas, e estudam a importância da ordem das instruções condicionais; 3) uma variante do Problema da Chuva [80], que tem sido um problema recorrente em pesquisas sobre educação em computação [28, 74]; e 4) um aplicativo de conversão de moedas usado em nossos contextos, onde os alunos trabalham com objetos, métodos e dicionários. Uma solução de exemplo para cada um dos quatro exercícios de programação é apresentada no Apêndice A.

Como o OpenAI Codex foi avaliado principalmente com a linguagem de programação Python em trabalhos anteriores [11] e, segundo relatos, funciona melhor em Python1, todos os nossos exercícios que usamos para preparar o OpenAI Codex são em Python. Em nossas explorações, usamos o modelo code-davinci-001 do Codex, que era a versão mais capaz (embora mais lenta) quando esses experimentos foram conduzidos.

3.2.2 Criação de exercícios de programação. Exploramos uma variedade de abordagens de preparação para a criação de exercícios de programação. No

¹Conforme observado na documentação do OpenAI Codex Beta, acessada pela última vez em 25/03/2022: <https://beta.openai.com/docs/engines/codex-series-private-beta>

Por fim, o priming que consideramos mais confiável para a criação de novos exercícios de programação continha uma descrição do problema, uma solução de exemplo e testes automatizados. Além disso, exploramos a adição de conceitos relacionados à programação (por exemplo, condicional, loop) e conceitos contextuais (por exemplo, caminhada, pesca) ao priming. Em geral, observamos que a introdução de conceitos levou o OpenAI Codex a levá-los em consideração ao criar exercícios de programação, embora os exercícios criados sem os conceitos também fossem significativos. Para ver como esses primings são formatados, consulte o Apêndice B. Além dos exemplos no Apêndice B, ao fornecer as amostras como entrada para o Codex, as amostras foram sufixadas com uma sequência de parada ("""). Após a sequência de parada, o priming incluía o texto "Exercício 2", os conceitos desejados no exercício criado e o identificador do enunciado do problema (--Enunciado do problema--). Um exemplo de um priming completo (ou seja, a entrada para o Codex) e um exemplo da saída correspondente gerada pelo Codex podem ser encontrados no Apêndice C.

Geramos exercícios utilizando os dois exercícios de preparação do Apêndice B, variando tanto os conceitos relacionados à programação quanto os conceitos contextuais (ver Tabela 1). Usando um total de nove conceitos contextuais (e um extra para omitir o conceito contextual) e dois conjuntos de conceitos relacionados à programação (e um extra para omitir os conceitos relacionados à programação), geramos um total de $10 \times 3 \times 2 = 60$ combinações diferentes de entradas (conceitos contextuais $\times$ conjuntos de conceitos relacionados à programação $\times$ preparações para os exercícios). Além disso, exploramos dois valores para o parâmetro de temperatura do Codex (0 e 0,75) e criamos dois exercícios para cada combinação de parâmetro. No total, isso resultou em uma amostra de $60 \times 2 \times 2 = 240$ exercícios de programação.

3.2.3 Criação de explicações de código. De forma semelhante à criação de exercícios de programação, exploramos diferentes tipos de abordagens de priming para a criação de explicações de código. Identificamos três tipos de primings que levaram a diferentes tipos de descrições de código: 1) uma descrição de alto nível do código, 2) uma descrição do código semelhante a um enunciado do problema e 3) uma explicação passo a passo do código. Neste trabalho, focamos no último tipo de explicação de código, ou seja, a explicação passo a passo do código, pois ela se alinha com o nível multiestrutural da taxonomia SOLO e é frequentemente produzida por alunos quando solicitados a explicar código [52].

Em nossos experimentos, o uso de um estímulo inicial composto pelo código-fonte, seguido por uma sequência de parada e o texto "Explicação passo a passo do programa acima:", e o número um seguido de um ponto, tendeu a produzir explicações passo a passo. Como exemplo, o estímulo inicial para um programa simples "Olá, mundo!" seria o seguinte:

```
imprimir ( " Olá rl d ! " )
```

Explicação passo a passo do programa acima:
1.

Com o preparo acima, o Codex criaria uma explicação passo a passo do código print("Olá, mundo!"). Para a análise da explicação passo a passo do código, criamos cinco explicações para cada uma das quatro soluções de exemplo dos exercícios de programação no Apêndice A, resultando em um total de 20 explicações de código. Como estávamos interessados em explicações precisas em vez de criativas, usamos o valor de temperatura 0 para gerar cada uma das explicações.

3.3 Avaliação

3.3.1 Exercícios de programação. A avaliação dos exercícios de programação foi realizada por meio de pesquisa de métodos mistos, na qual os exercícios foram avaliados tanto qualitativa quanto quantitativamente.

Na análise qualitativa, focamos em uma amostra aleatória de 120 exercícios de programação. Nosso foco foi na adequação, originalidade e prontidão para uso dos exercícios criados, conforme descrito na RQ1. Ao avaliar a adequação, analisamos se o exercício representa um problema relevante para os alunos – o enunciado descreve um problema prático que poderia ser proposto aos alunos para resolução? Ao avaliar a originalidade, verificamos se já existe uma cópia integral do exercício ou um exercício similar disponível online (utilizamos o Google e o GitHub para a busca). Relacionado à originalidade, também examinamos a atualidade dos exercícios – como os diferentes conceitos introdutórios são abordados nos exercícios criados? Ao avaliar a prontidão para uso, consideramos a quantidade de trabalho manual que um professor teria que realizar com os exercícios e os recursos associados.

solução de amostra e testes.

A análise qualitativa foi conduzida por quatro pesquisadores, que inicialmente realizaram a avaliação da sensatez, da novidade e da prontidão para uso em conjunto, discutindo os problemas identificados e os casos extremos.

A análise foi conduzida individualmente utilizando a rubrica descrita na Tabela 2, onde cada pesquisador trabalhou em uma subamostra dos exercícios de programação e avaliou os itens selecionados com afirmações de Sim / Não / Talvez, adicionando notas sempre que necessário. Todas as respostas com "Talvez" foram então analisadas em conjunto por pelo menos dois pesquisadores, trabalhando em paralelo para chegar a um consenso sobre se deveriam ser consideradas como Sim ou Não.

Em seguida, analisamos quantitativamente as respostas Sim / Não / Talvez e relatamos e discutimos os resultados. Para a análise quantitativa, exploramos três questões adicionais relacionadas à prontidão de uso dos exercícios, que foram calculadas a partir do conjunto total de 240 exercícios de programação. Essas questões estão descritas na Tabela 3 e as respostas foram obtidas programaticamente: 1) testamos se as soluções de exemplo podiam ser executadas, 2) testamos se a solução de exemplo passou nos testes automatizados e 3) verificamos a cobertura de instruções dos testes automatizados².

3.3.2 Explicações de código. De forma semelhante aos exercícios gerados, analisamos a capacidade do Codex de gerar explicações em linguagem natural para exemplos de código tipicamente encontrados em aulas introdutórias de programação.

Analisamos as 20 explicações de código geradas, inspecionando os tipos de erros presentes e sua frequência nas explicações dos diferentes programas de priming. Ao analisar as explicações de código, respondemos à pergunta "Todas as partes do código estão explicadas?" (Sim / Não) e contabilizamos a proporção de linhas explicadas corretamente em relação ao total de linhas de explicação geradas.

Foi viável para os quatro pesquisadores avaliarem colaborativamente todas as explicações em análise. Discutimos cada explicação gerada individualmente e desenvolvemos um entendimento compartilhado sobre o que significava uma única linha dentro de uma explicação estar correta. Decidimos ser bastante rigorosos em nossa avaliação para não influenciar artificialmente as explicações.

²A análise da cobertura de instruções dos testes automatizados foi realizada usando o Coverage.py versão 6.3.2 (<https://coverage.readthedocs.io/>).

Tabela 1: Palavras-chave utilizadas para a geração de exercícios de preparação. Os conceitos relacionados à programação são agrupados em dois conjuntos para reduzir a complexidade. o número de combinações possíveis.

Conceitos contextuais relacionados à programação: conjunto de conceitos 1: "função"; conjunto de conceitos 2: "classe"		
caminhadas, pesca, relacionamentos, futebol, música, saúde, hóquei no gelo, livros, culinária	função dicionário de parâmetros, compreensão de dicionário, aritmética	aula lista compreensão de listas condicional

Tabela 2: Rubrica de avaliação manual

Aspecto	Pergunta	Opções
Sensibilidade	O enunciado do problema descreve um problema plausível?	Sim / Não / Talvez
Novidade	Não conseguimos encontrar o exercício de programação por meio de uma busca online. (Google e GitHub) da declaração do problema?	Sim / Não / Talvez
Prontidão: problema e solução	O enunciado do problema corresponde à solução do modelo?	Sim / Não / Talvez
Tópico: função / classe	O enunciado do problema se refere a uma função ou classe quando esse conceito é fornecido como um conceito introdutório?	Sim / Não / Talvez
Tópico: lista / dicionário	O enunciado do problema incorpora uma lista ou um dicionário? Esse conceito é apresentado como um conceito introdutório?	Sim / Não / Talvez
Atualidade: contexto	O tópico do problema apresentado corresponde ao contexto fornecido?	Sim / Não / Talvez
Notas de formato livre	Notas	Texto livre

Tabela 3: Rubrica de avaliação automatizada

Aspecto	Pergunta	Responder
Prontidão: capacidade de execução da solução	Podemos executar a solução de exemplo sem erros?	Sim / Não / Não se aplica
Prontidão: solução e testes	A solução de exemplo passa nos testes unitários?	Sim / Não / Não se aplica
Prontidão: cobertura de testes	Em que medida os testes unitários abrangem a solução do modelo (declaração)? cobertura)?	0 a 100% / N.A.

relatar resultados mais fortes e exigir que a linguagem em cada linha fosse alterada para Seja preciso. Por exemplo, consideramos que uma explicação estava incorreta. se estivesse escrito "menor ou igual a x", onde o código correspondente... estava verificando "menor que x". Da mesma forma, se houvesse ambiguidade quanto a se a parte "else" na explicação de um "elif" foi considerada pois consideramos isso incorreto. Por exemplo, em um FizzBuzz Em um programa, uma linha como "elif number % 3 == 0:" seria classificada como como incorreta se a explicação da linha começar diretamente com "se" o número é divisível por 3" e não tentou qualificar o descrição com "caso contrário" ou frase similar para denotar sua lógica relação com o "se" correspondente. Optamos por ser lenientes apenas no caso em que as explicações não mencionaram explicitamente a inicialização de variáveis, embora se possa argumentar que isso seja relevante. em uma explicação abrangente.

4 RESULTADOS

4.1 Exercícios de Programação

No total, selecionamos e avaliamos aleatoriamente 120 dos 240 exercícios de programação criados pelo OpenAI Codex. A avaliação dos exercícios de programação incluiu a análise de sua sensatez, originalidade, prontidão, e também anotar quaisquer observações adicionais durante o processo. Além disso, para todos os 240 exercícios de programação, nós avaliou programaticamente se as soluções de amostra poderiam ser executado, se os testes automatizados foram aprovados e calculado o Cobertura das declarações dos testes automatizados.

As estatísticas relativas à sensatez, novidade e prontidão dos exercícios de programação avaliados são apresentadas na Tabela 4. Destes, 75,0% eram sensatos, 81,8% eram inovadores. , e 76,7% tinham uma correspondência solução de exemplo. As anotações em formato livre discutiam principalmente questões que incluía a existência de informações redundantes, informações faltantes, Valores ausentes ou incorretos nas entradas e/ou saídas de amostra (alguns)

3Observe que, segundo nossa definição de novidade, enunciados de problemas sem sentido são bastante...
Certamente classificado como novidade.

Tabela 4: Resumo dos exercícios de programação avaliados manualmente. Um exercício é considerado sensato se os requisitos forem descritos claramente dentro de um contexto lógico, inovador se a descrição do exercício não retornar resultados válidos quando usada como entrada para uma busca no Google ou GitHub, e possui uma solução de exemplo correspondente se a solução de código gerada corresponder à descrição.

Exercícios Sensible Novel Matches exemplo de solução Matches priming tópico Matches priming conceito Matches priming conceito função/classe						
						lista/dicionário
120	75,0%	81,8%	76,7%	79,2%	78,3%	75,8%

Tabela 5: Resumo da análise programática dos exercícios de programação gerados

	Possui solução de exemplo?	É possível executar a solução de exemplo?	Possui testes?	Todos os testes foram aprovados?	Cobertura de testes
Percentagens	84,6%	89,7%	70,8%	30,9%	98,0%
n em N	203 / 240	182 / 203	1701 / 240	51 / 1651	482 / 51

¹Cinco dos exercícios gerados continham --Testes-- mas não --Solução de exemplo-- (necessária para a extração automática das partes do conteúdo).
20 valor de n em N para a cobertura de teste é contabilizado como o número de casos de cobertura total (100%) em relação ao número total de conjuntos de testes que não falharam (ou seja, quando a cobertura pode ser calculada).

Os autores dos testes abordaram as questões que envolviam a formulação do problema (incluindo exemplos de entradas e saídas), discutiram as discrepâncias entre a formulação do problema e um exemplo de solução, e descreveram os motivos pelos quais os testes automatizados não seriam aprovados.

As estatísticas da análise programática realizada em todos os 240 exercícios de programação criados são apresentadas na Tabela 5. Dos 240 exercícios, 2034 possuíam uma solução de exemplo (84,6%). Das 203 soluções de exemplo, 182 (89,7%) puderam ser executadas (ou seja, a execução do código não produziu erros). Um total de 1704 exercícios de programação possuíam testes automatizados, enquanto 165 exercícios possuíam tanto uma solução de exemplo quanto testes automatizados. Desses 165 exercícios, 51 possuíam uma solução de exemplo que passou nos testes automatizados. Dos 51 exercícios com uma solução de exemplo funcional e testes automatizados, 48 apresentaram 100% de cobertura de instruções, e a cobertura média de instruções em todos os 51 exercícios foi de 98,0%. Ao inspecionar as anotações dos exercícios com testes automatizados que não passaram nos testes, observamos que o problema mais comum não estava relacionado à lógica do código, mas sim à forma como as saídas eram tratadas. Nesses casos, a solução de exemplo imprimia um valor, enquanto os testes automatizados esperavam que a solução de exemplo retornasse um valor (por exemplo, os testes chamavam uma função e esperavam que a função retornasse um valor, mas a função imprimia um valor). Observamos, é claro, que a confusão entre imprimir e retornar valores é um erro comumente citado cometido por iniciantes [24, 36]. Além disso, um problema comum era que os testes esperavam números específicos que não eram possíveis com as entradas (por exemplo, ao verificar se um programa extraía e retornava corretamente uma lista de valores pares de uma lista recebida como parâmetro, um teste fornecia a lista [1, 2, 3] como entrada para a função e esperava que a função retornasse a lista [2, 4]).

Nos resultados da análise programática sobre prontidão, apresentados na Tabela 5, vemos que em cerca de 90% das vezes as soluções de exemplo geradas são código válido e executável, os testes são gerados e autoextraíveis em aproximadamente 70% das vezes, enquanto apenas cerca de 30%

Das soluções geradas, 48 passaram nos testes (o que requer tanto uma solução sólida quanto testes sólidos). Surpreendentemente, quando os testes gerados passaram, obtivemos, em média, 98% de cobertura de testes, e 48 dos 51 conjuntos de testes aprovados cobriram 100% das declarações da solução de exemplo. Além disso, observamos que, em vários casos, apenas pequenos ajustes seriam necessários para transformar testes reprovados em aprovados. Nos casos em que faltavam testes, bastava adicionar o exercício gerado à preparação inicial, e os testes provavelmente seriam gerados em uma "segunda" execução (testamos esse comportamento diretamente ao explorar a saída).

4.2 Explicações do Código Um

total de 20 explicações de código criadas pelo OpenAI Codex a partir do código-fonte disponível no Apêndice A foram analisadas em conjunto pelos pesquisadores. Ao avaliar as explicações do código, verificamos se todas as partes do código foram explicadas e se cada linha foi explicada corretamente. A Tabela 6 apresenta as estatísticas da análise. Das 20 explicações de código, 90% explicaram todas as partes do código. No total, as explicações do código continham 174 explicações linha por linha, das quais 117 estavam corretas (67,2%).

As explicações incorretas estavam principalmente relacionadas à interpretação incorreta de comparações e condicionais de ramificação. Por exemplo, o Codex frequentemente explicava ``speed > 100`` como "se a velocidade for menor que 100" ou ``elif number % 3: `` como "se o número for divisível por três". Encontramos esses problemas consistentemente em cada uma das explicações geradas para duas das quatro amostras de código usadas para as explicações iniciais. Outra linha incorreta recorrente, embora menos persistente, incluía a frase "o programa termina se o usuário inserir dados", quando, na verdade, um laço ``while`` era finalizado e o programa ainda executava as linhas restantes após o laço. Notavelmente, para o quarto dos nossos códigos iniciais, aquele que continha uma classe de conversor de moedas e o uso dessa classe, nenhuma das cinco explicações geradas continha linhas incorretas e as explicações abrangiam todas as partes do programa.

4A contagem real é ligeiramente maior, uma vez que esse valor é calculado a partir de conteúdo extraível programaticamente (requer a palavra-chave relevante envolvida por dois traços "--" provenientes do início do conteúdo gerado).

Tabela 6: Resultados da explicação do código

Explicações do código: Todas as partes do código explicadas; Total de linhas; Linhas explicadas corretamente.			
20	90%	174	117 (67,2%)

5 DISCUSSÃO

5.1 Exercícios de Programação

A maioria dos exercícios gerados pareceu ser sensata e inovadora, incluindo uma solução de exemplo executável. Bem menos impressionante foi a qualidade dos conjuntos de testes e o desempenho do código em relação a esses testes. Apenas cerca de 70% dos exercícios incluíam testes, e destes, o conjunto de testes foi aprovado em menos de um terço dos casos. No entanto, na prática, pode ser possível contornar essa deficiência de duas maneiras. Primeiramente, instruímos o Codex a gerar todas as partes do exercício de programação em uma única etapa de saída. A demonstração do Codex feita pela OpenAI ilustrou um desempenho particularmente bom ao trabalhar interativamente com o Codex, guiando-o passo a passo. Portanto, poderíamos ter obtido mais sucesso na geração de bons casos de teste se os solicitássemos explicitamente. Isso poderia ser alcançado fornecendo uma descrição do problema e uma solução de exemplo como entrada para o Codex e configurando-o para gerar apenas os testes. Testamos isso na prática usando o Codex para criar com sucesso testes para alguns exercícios de programação criados com o Codex. Em segundo lugar, dado que é possível automatizar a verificação dos testes executando a solução de exemplo, simplesmente regenerar uma saída repetidamente até que um conjunto de testes bem-sucedidos seja produzido pode ser uma estratégia válida na prática.

Embora nosso foco principal neste artigo tenha sido a geração de exercícios e sua prontidão para uso, acreditamos que há valor mesmo naqueles exercícios que podem ser aprimorados. Em particular, eles podem servir de inspiração para instrutores que podem facilmente modificar os problemas manualmente, e podem até mesmo formar a base para novos tipos de atividades para os alunos. Por exemplo, muitos educadores apreciarão a redução da frustração de ter que escrever "mais um" problema de prática ou questão de prova, e isso levou a um trabalho colaborativo em torno do compartilhamento de exercícios de programação [22, 34]. A facilidade com que novos exercícios podem ser gerados com uma ferramenta como o Codex, mesmo que os exercícios resultantes não sejam usados literalmente, pode ajudar os instrutores a gerar ideias rapidamente e superar o bloqueio criativo típico dos educadores da área de computação. Afinal, modificar um exercício de programação existente é mais fácil do que escrever um do zero. Outro problema que observamos, mesmo em exercícios gerados que eram inovadores, sensatos e tinham testes, foi que às vezes eles eram pouco especificados. Ou seja, a descrição do problema não especificava explicitamente como os casos extremos deveriam ser tratados. Um bom exemplo disso é a classe 'Fisherman' apresentada logo no início deste artigo. A descrição do problema não indica o que deve acontecer se a contagem de um determinado tipo de peixe chegar a zero quando o método `throw_away()` for chamado. Nesse caso, a solução de exemplo para o problema fornece a resposta: o tipo de peixe deve ser removido do dicionário (em vez de permanecer e ser exibido com o valor zero). Problemas como esse podem ser um bom ponto de partida para discussões entre os alunos sobre o tema.

testes de programa, e podem servir de base para novas tarefas em que os alunos devem aprimorar as especificações do problema.

Um aspecto da geração de exercícios que nos surpreendeu particularmente foi a forma como os conceitos contextuais e os conceitos relacionados à programação foram incorporados às novas descrições de problemas. A Tabela 7 mostra as descrições dos problemas para dois dos exercícios gerados com o Codex. Cada exercício foi gerado a partir de um problema de programação diferente (ver Apêndice B), mas apresentam pouca semelhança com esses problemas. Os enunciados dos problemas gerados não eram apenas variações triviais (como alterações gramaticais) dos exercícios de priming, mas eram materialmente diferentes e incorporavam os temas contextuais de forma bastante natural, como calcular uma lista de amigos ou calcular a variação de altitude de um excursionista para os temas 'relacionamento' e 'caminhada', respectivamente, conforme mostrado na Tabela 7. Os exercícios gerados usando o tema contextual de 'livros' incluíam casos de teste envolvendo títulos e autores populares como 'Ender's Game', 'Rainbow Six' e 'JRR Tolkien'; o tema 'futebol' resultou em testes envolvendo 'Lionel Messi' e 'Cristiano Ronaldo'; e o tema 'saúde' resultou em exercícios onde fumar cigarros e comer maçãs foram contrastados como atividades não saudáveis e saudáveis, respectivamente.

Essa capacidade de contextualizar automaticamente enunciados de problemas pode ter aplicações práticas úteis. Para professores, oferece o potencial de gerar exercícios de programação que visam construções específicas e exigem certos tipos de soluções. Para alunos, trabalhos anteriores que exploraram o efeito da descrição do problema no ensino da computação mostraram que um contexto familiar dentro da narrativa de um problema pode facilitar o aprendizado. A formulação do problema pode ter um efeito positivo no desempenho [44]. Não é possível para um professor selecionar contextos apropriados que sejam familiares e de interesse para todos os alunos, especialmente considerando a diversidade de experiências em grandes turmas do primeiro ano. Nossos resultados sugerem que pode ser possível que alunos individualmente forneçam suas próprias palavras-chave e tenham exercícios personalizados gerados para seu uso pessoal, possivelmente utilizando exercícios criados pelo professor como estímulo. Explorar isso com mais detalhes, e em particular coletar as opiniões dos alunos sobre a adequação de questões personalizadas em comparação com conjuntos de problemas mais genéricos, é uma via fascinante para trabalhos futuros.

5.2 Explicações de Código

Neste artigo, nossa investigação sobre explicações de código geradas com o Codex focou em descrições linha por linha do código. Como discutido na Seção 2.2, esses tipos de descrições se alinham com o nível multiestrutural da taxonomia SOLO e são comumente produzidos por alunos quando solicitados a explicar código, especialmente alunos com baixo desempenho que mais se beneficiariam de alguma ajuda com a explicação e reflexão sobre o código [52]. As explicações geradas pelo Codex foram bastante completas, pois todas as partes essenciais do código geralmente eram abordadas na explicação, mas frequentemente continham pequenas imprecisões. Isso levanta questões sobre a utilidade das explicações para ajudar os alunos a entender ou depurar seus próprios códigos.

5<https://openai.com/blog/openai-codex/>

Tabela 7: Exemplos de enunciados de problemas onde conceitos contextuais (relacionamentos e caminhadas) e conceitos de programação (classe e função) foram incorporados com sucesso. O código-fonte dos primos ('speeding_check' e 'Converter') pode ser encontrado no Apêndice B. Espaços em branco e outras formatações estruturais foram removidos por questões de espaço.

Prime: 'verificação de velocidade' Conceito contextual: 'relacionamentos' Conceito de programação: 'classe'	Prime: 'Conversor' Conceito contextual: 'caminhada' Conceito de programação: 'função'
Crie uma classe chamada `Pessoa` que contenha uma lista de amigos. Ela deve ter métodos para adicionar e remover amigos. Escreva uma função chamada `encontrar_amigos` que receba um único parâmetro chamado `pessoa` e que liste os amigos dessa pessoa. Use a classe `Pessoa` para criar duas pessoas e adicionar amigos a elas. Imprima todos os amigos da primeira pessoa.	Escreva uma função chamada `hiking` com os seguintes parâmetros: `elevation_chart` é um dicionário contendo a altitude em metros de vários locais no mundo; `path` é uma lista de tuplas, onde cada tupla contém dois nomes (strings) de locais no mapa. O primeiro nome é o local onde o caminho começa e o segundo nome é o local onde o caminho termina; `uphill_hiking` é um número que representa o quanto o caminhante está disposto a subir. Em outras palavras, é a porcentagem máxima de altitude que o caminhante está disposto a escalar; `downhill_hiking` é um número que representa o quanto o caminhante está disposto a descer. Em outras palavras, é a porcentagem máxima de altitude que o caminhante está disposto a escalar. A função deve calcular a distância vertical total percorrida pelo caminhante no caminho e retornar a distância em metros.

código. No entanto, como um estímulo para uma discussão entre um assistente de ensino e um aluno, as explicações geradas ainda podem fornecer um ponto de partida útil. Outros tipos de andaimes tecnológicos para apoiar conversas presenciais entre alunos e professores, como a ferramenta Pensieve [94], provaram ser valiosos.

Preparamos explicitamente o Codex para produzir explicações em nível multiestrutural usando uma instrução que solicitava uma "explicação passo a passo" do código e terminava com o item inicial da lista enumerada (ou seja, '1.'). Uma questão interessante relacionada é até que ponto o Codex consegue gerar explicações de nível superior, no nível relacional da taxonomia SOLO, que abstraíram com sucesso o propósito do código. Ao experimentar com muitos outros tipos de instruções de preparação para gerar as explicações, descobrimos que o Codex raramente fornecia descrições de alto nível. Isso corrobora as descobertas de Kojima et al., que constataram que grandes modelos de linguagem parecem ter um desempenho melhor em tarefas de raciocínio quando são preparados para "pensar passo a passo" [42]. Mesmo uma instrução muito explícita, como "Uma descrição de alto nível do programa acima:", ainda resultaria normalmente em uma explicação linha por linha. Um exemplo disso é a seguinte explicação gerada em resposta ao programa 'speed-ing_check', que também ilustra um erro sutil, pois os casos em que a entrada é igual a 120 ou 100 não são considerados corretamente:

- A função speed_check verifica a velocidade do carro e retorna o resultado esperado. Primeiro, a função compara o valor da velocidade com 120 e, se o valor for maior que 120, retorna "Você foi multado em \$200". Se o valor da velocidade for maior que 100, mas menor que 120, retorna "Você foi multado em \$100" e, se o valor for menor que 100, retorna "Tudo certo, siga em frente".

Ocasionalmente, observamos respostas em um nível mais abstrato. Vários exemplos são ilustrados na Tabela 8. Os exemplos A e B mostram respostas de nível relacional aos prompts 'speeding_check' e 'fizz_buzz'. O exemplo C inclui algumas informações de contexto interessantes sobre o problema FizzBuzz. O exemplo D é uma saída sem sentido que foi gerada uma única vez em resposta ao prompt do problema Rainfall.

5.3 Trabalho Futuro

Vemos um grande potencial para trabalhos futuros utilizando o Codex e outros modelos similares no contexto do ensino de programação. Dados os resultados positivos que observamos em termos de geração de exercícios de programação, temos interesse em desenvolver um gerador de exercícios automatizado, baseado no Codex, que possa ser utilizado por instrutores.

A ferramenta poderia fornecer uma camada de validação sobre o Codex, permitindo que os professores filtrassem quaisquer questões que não incluam soluções de exemplo válidas ou um conjunto abrangente de testes precisos. No presente trabalho, nosso foco foi em exercícios introdutórios de programação, mas seria interessante explorar a geração de exercícios de maior complexidade. Por exemplo, investigar se o Codex é capaz de gerar especificações precisas para tarefas ou projetos maiores , ou para aqueles relacionados a conceitos de computação mais avançados.

Com relação às explicações do código, trabalhos futuros devem explorar se elas podem ser usadas como base para gerar questões de múltipla escolha relacionadas ao código do próprio aluno, similar a trabalhos anteriores [43], que poderiam servir como uma tarefa de reflexão. Por exemplo, seria possível criar uma explicação do programa do aluno, bem como várias outras explicações para pequenas modificações nesse programa, similar em metodologia ao teste de mutação [38] (por exemplo, com operadores relacionais invertidos). Esse conjunto de explicações poderia então ser mostrado ao aluno, cabendo a ele selecionar a explicação que melhor corresponde ao seu código. De maneira similar, as explicações criadas com o Codex poderiam ser transformadas em problemas de Parsons [18], por exemplo, onde cada linha de uma explicação linha por linha é apresentada ao aluno em ordem aleatória para que ele a decifre.

Embora tenhamos observado imprecisões nas explicações do código geradas neste estudo, que podem limitar essas ideias por enquanto, é provável que modelos como o Codex continuem a melhorar com o tempo.

Neste trabalho, analisamos qualitativamente as explicações de código criadas com o Codex. Trabalhos futuros devem explorar como essas explicações podem ser usadas pelos alunos na prática, por exemplo, fazendo com que os alunos avaliem a qualidade e a utilidade das explicações criadas . Uma abordagem instrucional que se tornou cada vez mais comum no ensino de computação é o learnersourcing [41] , onde os alunos participam da criação e avaliação de materiais do curso, como questões e exercícios (ver, por exemplo, [14, 16, 47, 65]).

Geração automática de exercícios de programação e explicações de código usando grandes modelos de linguagem. ICER 2022, 7 a 11 de agosto de 2022, Lugano e evento virtual, Suíça.

Tabela 8: Exemplos de explicações de código incomuns produzidas em resposta aos problemas speeding_check (A) e FizzBuzz (B, C). (D) é um exemplo de uma explicação sem sentido para o problema Rainfall.

A	função A recebe a velocidade do carro como entrada. Verifica a velocidade e imprime uma multa de acordo com a velocidade do condutor.
B	é um programa simples que verifica se um número é divisível por 3, 5 ou ambos.
C	Este programa é uma variação do FizzBuzz, um desafio de programação popular. Ele pode ser encontrado aqui: https : //blog.codinghorror.com/why-cant-programmers-program/ . O programa acima lê um array de inteiros, realiza uma verificação condicional if-else nos números e imprime o resultado.
D	possui várias variáveis de pesquisa globais e locais. Possui um nó inicial e um nó final. Possui diversas variáveis, como nomes de variáveis, que são usadas para acessar os valores e para agrupar a lista de métodos.

Por exemplo, a ferramenta Quizius, descrita por Saarinen et al., permite que os alunos contribuam com perguntas para um repositório, e suas respostas são usadas para produzir estimativas estatísticas da prevalência de concepções errôneas sobre o tópico [70]. Uma nova abordagem para a aprendizagem colaborativa poderia fazer com que os alunos se concentrassem na avaliação de artefatos criados pelo Codex. Prevemos um novo tipo de aprendizagem colaborativa que denominamos “robótica colaborativa”, onde modelos de aprendizado de máquina semelhantes aos do Codex são usados para criar automaticamente artefatos similares à colaboração coletiva tradicional, mas onde esses materiais de aprendizagem “robóticos” são então avaliados pelos alunos. Isso resolveria um dos principais desafios relacionados ao uso de recursos de aprendizagem, que é o fato de os alunos tenderem a estar muito mais inclinados a usar e avaliar recursos criados por outros do que a criar recursos eles mesmos [66, 78].

Existem também aplicações óbvias do Codex que não avaliamos neste trabalho, mas que têm implicações importantes para o ensino da computação. Em particular, o preenchimento automático em tempo real e a geração automática de código-fonte existente. Um uso potencial do Codex no ensino de programação poderia ser uma ferramenta semelhante ao GitHub Copilot6, que apresenta aos alunos dicas ou sugestões para melhorias no código. A ferramenta permitiria que o instrutor ajustasse esse feedback de uma forma pedagogicamente adequada, em vez de liberar todo o poder dessas ferramentas para os alunos. Essa linha de pesquisa se alinha ao caminho Aluno y Tentativa no modelo que apresentamos na Figura 1.

Por fim, a combinação do GPT-3 e do Codex poderia facilitar todo o processo de criação de material didático. Para sermos claros, acreditamos ser improvável que grandes modelos de linguagem como o GPT-3 e o Codex possam substituir completamente os professores como criadores de material de aprendizagem. No entanto, como o desenvolvimento no processamento de linguagem natural é rápido e as capacidades dos modelos continuam a melhorar, é possível que, num futuro próximo, um instrutor possa agilizar a criação tanto de materiais textuais quanto de exercícios de programação por meio de instruções cuidadosamente elaboradas para esses modelos, em que a saída dos modelos precisaria apenas de pequenas alterações antes de ser disponibilizada aos alunos.

5.4 Ameaças à Validade Existem

algumas ameaças à validade deste trabalho, que discutiremos aqui. Primeiramente, em relação à nossa análise qualitativa dos exercícios de programação e explicações de código criados pelo Codex, tínhamos um conjunto relativamente pequeno de exemplos criados e exploramos apenas alguns tipos diferentes de instruções: quatro exercícios diferentes para explicações de código e dois exercícios diferentes como instruções na criação de código.

6<https://copilot.github.com/>

novos exercícios. É possível que diferentes instruções pudessem ter levado a resultados de qualidade diferente, e uma avaliação de uma variedade maior de insumos é justificada.

Além disso, em nossa análise qualitativa dos exercícios de programação criados, não calculamos a confiabilidade interavaliadores. No entanto , os pesquisadores trabalharam em estreita colaboração em um subconjunto das avaliações e discutiram todos os casos ambíguos, abordando parcialmente essa questão.

preocupação.

Ass considerarmos a novidade dos exercícios de programação criados, pesquisamos possíveis correspondências tanto no Google quanto no GitHub. É possível que esta análise tenha deixado de fora algumas fontes, como sites protegidos por senha que não são indexados pelo Google. Também é possível que alguns repositórios usados pelo Codex durante o treinamento (e dos quais o Codex poderia tecnicamente produzir conteúdo verbatim) tenham sido excluídos ou tornados privados entre o momento em que o Codex foi ajustado e nossa análise. No entanto, consideramos essa possibilidade muito remota [12]. Além disso, nossa definição de novidade baseou-se principalmente no fato de os exercícios serem inovadores no sentido de não serem cópias diretas de exercícios existentes. Trabalhos futuros devem estudar a novidade com uma definição mais ampla, por exemplo, investigando se o Codex combina conceitos de programação de maneiras inovadoras.

Uma possível questão relacionada à generalização dos nossos resultados é que nos concentramos na criação de exercícios de programação e explicações de código em inglês. É possível que a criação desses materiais em outros idiomas seja mais difícil (por exemplo, que os exercícios criados sejam mais propensos a serem sem sentido). Para abordar essa preocupação, realizamos uma breve análise da capacidade do Codex em criar exercícios em finlandês, um idioma com aproximadamente 5,8 milhões de falantes nativos e que é a língua materna de três dos autores.

Com base nessa breve análise, os exercícios criados foram considerados sensatos e a linguagem do texto que os acompanhava (por exemplo, a formulação do problema) era, em geral, boa.

Ao considerar o desempenho do Codex na resolução de problemas de programação [27], uma questão que pode surgir é se qualquer valor acrescentado por esta ferramenta para o instrutor será imediatamente anulado pelo seu uso pelos alunos para plágio. No entanto, os alunos poderão usar estes tipos de modelos independentemente do trabalho que explore os potenciais benefícios. Além disso, outras áreas, como a matemática e a física, têm sofrido com o problema de exercícios com resolução automática há décadas [37] – também é prática comum nessas disciplinas fornecer soluções para os problemas no final dos livros didáticos. A capacidade de resolver exercícios automaticamente ou ter as soluções disponíveis não impede que aqueles que desejam aprender o façam.

Reconhecemos que modelos de linguagem de grande porte demonstraram sofrer de vieses semelhantes aos dos humanos [73]; isso é esperado, visto que foram treinados com dados gerados por humanos. Assim, é possível que, por exemplo, o uso desses modelos para a criação de exercícios possa levar a exercícios que perpetuem esses vieses. Acreditamos que a abordagem com intervenção humana é essencial para moderar tais vieses ao utilizar modelos de linguagem de grande porte para gerar materiais de aprendizagem.

Por fim, analisamos os exercícios criados principalmente sob a ótica da abordagem pedagógica de "muitos exercícios pequenos" e não exploramos, por exemplo, a criação de exercícios de programação mais complexos. Assim, permanece desconhecido se o Codex é aplicável em contextos com exercícios mais extensos. Da mesma forma, estudamos apenas exercícios em Python – é possível que o Codex não seja tão eficiente na criação de novos exercícios em outras linguagens de programação, visto que foi relatado que o Codex é mais eficiente em Python 7.

6 CONCLUSÃO

Neste trabalho, exploramos em que medida o OpenAI Codex poderia 1) auxiliar instrutores na criação de exercícios de programação e 2) gerar explicações úteis de código-fonte. Estudamos isso por meio de duas questões de pesquisa, que respondemos a seguir: RQ1: Em que medida os exercícios de

programação criados usando o OpenAI Codex são sensatos, inovadores e facilmente aplicáveis?

A: Constatamos que a maioria dos exercícios de programação criados pelo Codex eram pertinentes, inovadores e incluíam uma solução de exemplo apropriada. Além disso, observamos que tanto o tema programático quanto o contexto dos exercícios criados podiam ser facilmente influenciados. Esse resultado sugere que o Codex pode, de fato, ser uma ferramenta útil para instrutores, facilitando o processo de criação de exercícios. Observamos, no entanto, que os exercícios de programação raramente estavam em um estado que permitisse adicioná-los diretamente a um curso, sem ajustes. Em particular, os enunciados dos problemas nem sempre abordavam casos extremos e muitos exercícios não possuíam testes ou apresentavam testes com falhas. Constatamos que os casos extremos poderiam ser facilmente adicionados por um professor, ou sua adição poderia ser transformada em uma atividade de aprendizagem. Da mesma forma, no caso dos testes ausentes, observamos que os testes podem ser facilmente gerados com o Codex, e muitos dos testes com falhas estavam relacionados a problemas fáceis de corrigir (por exemplo, somando um número ou retornando um valor em vez de imprimi-lo).

RQ2: Quão abrangentes e precisas são as explicações em linguagem natural das soluções de código para exercícios introdutórios de programação no OpenAI Codex?

A: Nossos resultados sugerem que as explicações geradas pelo Codex abrangem a maioria (90%) do código, embora contenham algumas imprecisões (67,2% das linhas de explicação estavam corretas). Observamos que, na maioria dos casos, as linhas com erros continham apenas pequenos equívocos que poderiam ser facilmente corrigidos por um instrutor ou por monitores. Avaliar o valor prático dessas explicações seria um trabalho futuro interessante, por exemplo, para verificar se elas poderiam ser utilizadas por monitores para agilizar o processo de auxílio a programadores iniciantes.

Em resumo, nossos resultados corroboram descobertas anteriores de que grandes modelos de linguagem são aprendizes zero-shot [42] e few-shot [9], o que significa que eles têm um bom desempenho em tarefas mesmo quando não recebem nenhum exemplo relacionado à tarefa como entrada, ou recebem apenas alguns. Nosso trabalho sugere que modelos modernos de aprendizado de máquina, como o OpenAI Codex, oferecem muitas oportunidades para os criadores de cursos de programação, embora os desafios potenciais descritos em trabalhos anteriores [27] não devam ser ignorados. Nossa análise atual mostrou resultados notáveis na criação de exercícios de programação inovadores e coerentes com soluções de exemplo prontas e testes automatizados, apesar da presença de alguns problemas de precisão e qualidade (que poderiam ser facilmente corrigidos manualmente). Também observamos resultados promissores nas explicações de código geradas. Prevemos que as possibilidades oferecidas pelos modelos generativos para a prática e a pesquisa em educação computacional só irão melhorar com o tempo, à medida que esses modelos continuam a evoluir.

AGRADECIMENTOS

Agradecemos à OpenAI por nos permitir o acesso ao Codex através do beta privado.

REFERÊNCIAS

[1] Onni Aarne, Petrus Peltola, Juho Leinonen e Arto Hellas. 2018. Um estudo sobre o prazer e a participação na programação em pares usando métricas de motivação e estratégia de estudo. In Anais do 49º Simpósio Técnico da ACM sobre Educação em Ciência da Computação. 759–764.

[2] Kirsti M Ala-Mutka. 2005. Um levantamento de abordagens de avaliação automatizada para tarefas de programação. Educação em ciência da computação 15, 2 (2005), 83–102.

[3] Ibrahim Albluwi. 2019. Plágio em Avaliações de Programação: Uma Revisão Sistemática. ACM Trans. Comput. Educ. 20, 1, Artigo 6 (dez. 2019), 28 páginas. <https://doi.org/10.1145/3371156>

[4] Joe Michael Allen, Frank Vahid, Kelly Downey e Alex Daniel Edgcomb. 2018. Programas semanais em uma aula de CS1: Experiências com programas de múltiplos níveis com correção automática (MSP). Em 2018, na Conferência e Exposição Anual da ASEE.

[5] Cory Althoff. 2022. O programador autodidata: o guia definitivo para programar profissionalmente. Hachette UK.

[6] Albert Bandura. 1977. Autoeficácia: rumo a uma teoria unificadora do comportamento mudança. Revisão psicológica 84, 2 (1977).

[7] Elisa Baniassad, Lucas Zamprogno, Braxton Hall e Reid Holmes. 2021. PARE A INSANIDADE (DO AUTOGRADADOR): Penalidades de regressão para deter a dependência excessiva do autograder. Em Anais do 52º Simpósio Técnico da ACM sobre Educação em Ciência da Computação (Evento Virtual, EUA) (SIGCSE '21). Association for Computing Machinery, Nova York, NY, EUA, 1062–1068. <https://doi.org/10.1145/3408877.3432430>

[8] John B. Biggs e KF Collis. 1982. Avaliando a qualidade da aprendizagem: a taxonomia SOLO (estrutura do resultado de aprendizagem observado) / John B. Biggs, Kevin F. Collis. Academic Press Nova Iorque. xiii, 245 p. : páginas.

[9] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever e Dario Amodei. 2020. Modelos de linguagem são aprendizes com poucos exemplos. Em Avanços em sistemas de processamento de informações neurais. 1877–1901.

[10] Binglin Chen, Sushmita Azad, Rajarshi Haldar, Matthew West e Craig Zilles. 2020. Uma Rubrica de Pontuação Validada para Perguntas de Explicação em Linguagem Simples. Association for Computing Machinery, Nova Iorque, NY, EUA, 563–569. <https://doi.org/10.1145/3328778.3366879> [11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Avaliando grandes modelos de linguagem treinados em código. arXiv preprint arXiv:2107.03374 (2021).

[12] Matteo Ciniselli, Luca Pascarella e Gabriele Bavota. 2022. Até que ponto os recomendadores de código baseados em aprendizado profundo geram previsões clonando código do conjunto de treinamento? arXiv preprint arXiv:2204.06894 (2022).

[13] Catherine H Crouch e Eric Mazur. 2001. Instrução por pares: Dez anos de experiência e resultados. American journal of physics 69, 9 (2001), 970–977.

[14] Paul Denny, Diana Cukierman e Jonathan Bhaskar. 2015. Medindo o efeito da criação de exercícios práticos na aprendizagem em um curso introdutório de programação. In Anais da 15ª Conferência Koli Calling sobre Educação em Computação

7Conforme observado na documentação do OpenAI Codex Beta, acessada pela última vez em 25/03/2022: <https://beta.openai.com/docs/engines/codex-series-private-beta>

Geração automática de exercícios de programação e explicações de código usando grandes modelos de linguagem. ICER 2022, 7 a 11 de agosto de 2022, Lugano e evento virtual, Suíça.

Pesquisa (Koli, Finlândia) (Koli Calling '15). Association for Computing Machinery, Nova York, NY, EUA, 13–22. <https://doi.org/10.1145/2828959.2828967> [15] Paul Denny, Andrew Luxton-Reilly, Ewan Tempero e Jacob Hendrickx. 2011. Codewrite: apoiando a prática de Java conduzida pelo aluno. In Anais do 42º Simpósio Técnico da ACM sobre Educação em Ciência da Computação. 471–476.

[16] Paul Denny, Ewan Tempero, Dawn Garbett e Andrew Petersen. 2017. Examinando uma atividade de perguntas geradas por alunos usando atribuição aleatória de tópicos. Em Anais da Conferência ACM de 2017 sobre Inovação e Tecnologia no Ensino de Ciência da Computação (Bolonha, Itália) (ITICSE '17). Association for Computing Machinery, Nova York, NY, EUA, 146–151. <https://doi.org/10.1145/3059009.3059033> [17] Iddo Drori, Sarah Zhang, Reece Shuttleworth, Leonard Tang, Albert Lu, Elizabeth Ke, Kevin Liu, Linda Chen, Sunny Tran, Newman Cheng, Roman Wang, Nikhil Singh, Taylor L. Patti, Jayson Lynch, Avi Shporer, Nakul Verma, Eugene Wu e Gilbert Strang. 2021. Uma rede neural resolve, explica e gera problemas de matemática universitária por meio de síntese de programas e aprendizado com poucos exemplos em nível humano. <https://doi.org/10.48550/ARXIV.2112.15594> [18] Yuemeng Du, Andrew Luxton-Reilly e Paul Denny. 2020. Uma revisão da pesquisa sobre problemas de

Parsons. Em Anais da Vigésima Segunda Conferência Australiana de Educação em Computação. 195–202.

[19] Angela Lee Duckworth e Lauren Eskreis-Winkler. 2013. Verdadeira garra. Aps Observer 26 (2013).

[20] Rodrigo Duran, Albina Zavgorodniaia e Juha Sorva. 2021. Teoria da Carga Cognitiva na Pesquisa em Educação em Computação: Uma Revisão. (2021). http://rodrigoduran.net/papers/CLT_in_CER.pdf Pré-publicação.

[21] John Edwards, Joseph Ditton, Dragan Trninic, Hillary Swanson, Shelsey Sullivan e Chad Mano. 2020. Exercícios de sintaxe em CS1. Em Anais da Conferência ACM de 2020 sobre Pesquisa Internacional em Educação em Computação. 216–226.

[22] Stephen H. Edwards, Jürgen Börstler, Lillian N. Cassel, Mark S. Hall e Joseph Hollingsworth. 2008. Desenvolvimento de um formato comum para compartilhamento de tarefas de programação. SIGCSE Bull. 40, 4 (nov. 2008), 167–182. <https://doi.org/10.1145/1473195.1473240>

[23] K Anders Ericsson, Ralf T Krampe e Clemens Tesch-Römer. 1993. O papel da prática deliberada na aquisição do desempenho especializado. Revisão psicológica 100, 3 (1993), 363.

[24] Andrew Ettles, Andrew Luxton-Reilly e Paul Denny. 2018. Erros de lógica comuns cometidos por programadores novatos. Em Anais da 20ª Conferência Australiana de Educação em Computação (Brisbane, Queensland, Austrália) (ACE '18). Associação para Maquinaria de Computação, Nova Iorque, NY, EUA, 83–89. <https://doi.org/10.1145/3160489.3160493> [25]

Katrina Falkner e Judy Sheard. 2019. Abordagens pedagógicas (1ª ed.). Cambridge University Press, Reino Unido, 445–480. <https://doi.org/10.1017/9781108654555.016>

[26] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. CodeBERT: Um modelo pré-treinado para programação e linguagens naturais. Pré-impressão do arXiv arXiv:2002.08155 (2020).

[27] James Finnie-Ansley, Paul Denny, Brett A Becker, Andrew Luxton-Reilly e James Prather. 2022. Os robôs estão chegando: explorando as implicações do OpenAI Codex na programação introdutória. Em Australasian Computing Education Conference. 10–19.

[28] Kathi Fisler. 2014. O problema da precipitação recorrente. In Anais do décimo Conferência anual sobre pesquisa internacional em educação em computação. 35–42.

[29] Max Fowler, Binglin Chen, Sushmita Azad, Matthew West e Craig Zilles. 2021. Correção automática de perguntas do tipo "Explique em linguagem simples" usando PNL. In Anais do 52º Simpósio Técnico da ACM sobre Educação em Ciência da Computação (Evento Virtual, EUA) (SIGCSE '21). Association for Computing Machinery, Nova York, NY, EUA, 1163–1169. <https://doi.org/10.1145/3408877.3432539>

[30] Brian Hanks, Sue Fitzgerald, Renée McCauley, Laurie Murphy e Carol Zander. 2011. Programação em pares na educação: uma revisão da literatura. Computer Science Education 21, 2 (2011), 135–173. <https://doi.org/10.1080/08993408.2011.579808> arXiv:<https://doi.org/10.1080/08993408.2011.579808> [31] Mohammed Hassan e Craig Zilles. 2021. Explorando questões de 'rastreamento reverso' como meio de avaliar a habilidade de rastreamento em exames de CS 1 baseados em computador. Em Anais da 17ª Conferência ACM sobre Pesquisa Internacional em Educação Computacional (Evento Virtual, EUA) (ICER 2021). Association for Computing Machinery, Nova York, NY, EUA, 115–126. <https://doi.org/10.1145/3446871.3469765> [32] John Hattie e Helen Timperley. 2007. O poder do feedback. Revisão de pesquisa educacional 77, 1 (2007), 81–112.

[33] Arto Hellas, Juho Leinonen e Petri Ihantola. 2017. Plágio em provas para fazer em casa: busca de ajuda, colaboração e trapaça sistemática. Em Anais da conferência ACM de 2017 sobre inovação e tecnologia no ensino de ciência da computação. 238–243.

[34] David Hovemeyer, Matthew Hertz, Paul Denny, Jaime Spacco, Andrei Papancea, John Stamper e Kelly Rivers. 2013. CloudCoder: Construindo uma comunidade para criar, atribuir, avaliar e compartilhar exercícios de programação (apenas resumo). In Anais do 44º Simpósio Técnico da ACM sobre Educação em Ciência da Computação (Denver, Colorado, EUA) (SIGCSE '13). Association for Computing Machinery, Nova York, NY, EUA, 742. <https://doi.org/10.1145/2445196.2445451>

[35] Petri Ihantola, Tuukka Ahoniemi, Ville Karavirta e Otto Seppälä. 2010. Revisão de sistemas recentes para avaliação automática de tarefas de programação. In Anais da 10ª Conferência Internacional Koli Calling sobre Pesquisa em Educação Computacional. 86–93.

[36] Cruz Izu e Peter Dinh. 2018. Programadores novatos conseguem escrever funções em C?. Em 2018 IEEE International Conference on Teaching, Assessment, and Learning for Engineering (TALE). 965–970. <https://doi.org/10.1109/TALE.2018.8615375> [37] MA Jenkins e Joseph Frederick Traub. 1967. Um algoritmo para um solucionador automático de polinômios gerais. CiteSeer.

[38] Yue Jia e Mark Harman. 2010. Uma análise e levantamento do desenvolvimento de testes de mutação. IEEE transactions on software engineering 37, 5 (2010), 649–678.

[39] Cazembe Kennedy, Aubrey Lawson, Yvon Feaster e Eileen Kraemer. 2020. Feedback entre pares baseado em concepções errôneas: uma técnica pedagógica para reduzir concepções errôneas. In Anais da Conferência ACM de 2020 sobre Inovação e Tecnologia no Ensino de Ciência da Computação (Trondheim, Noruega) (ITICSE '20). Association for Computing Machinery, Nova Iorque, NY, EUA, 166–172. <https://doi.org/10.1145/3341525.3387392> [40] Hieke Keuning, Johan Jeuring e Bastiaan Heeren. 2018. Uma revisão sistemática da literatura sobre geração automatizada de feedback para exercícios de programação. ACM Transactions on Computing Education (TOCE) 19, 1 (2018), 1–43.

[41] Juho Kim. 2015. Learnersourcing: melhorando a aprendizagem com a atividade coletiva do aluno. Tese de doutorado. Instituto de Tecnologia de Massachusetts.

[42] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo e Yusuke Iwasawa. 2022. Grandes modelos de linguagem são raciocinadores zero-shot. Pré-impressão do arXiv arXiv:2205.11916 (2022).

[43] Teemu Lehtinen, André L Santos e Juha Sorva. 2021. Vamos perguntar aos alunos sobre seus programas, automaticamente. Em 2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC). IEEE, 467–475.

[44] Juho Leinonen, Paul Denny e Jacqueline Whalley. 2021. Explorando os efeitos de descrições de problemas contextualizadas na resolução de problemas. In Australasian Computing Education Conference (Virtual, SA, Austrália) (ACE '21). Association for Computing Machinery, Nova York, NY, EUA, 30–39. <https://doi.org/10.1145/3441636.3442302>

[45] Juho Leinonen, Paul Denny e Jacqueline Whalley. 2022. Uma comparação de feedback imediato e agendado em projetos introdutórios de programação. Em Anais do 53º Simpósio Técnico da ACM sobre Educação em Ciência da Computação V. 1 (Providence, RI, EUA) (SIGCSE 2022). Association for Computing Machinery, Nova York, NY, EUA, 885–891. <https://doi.org/10.1145/3478431.3499372> [46] Juho Leinonen, Krista Longi, Arto Klami, Alireza Ahadi e Arto Vihavainen. 2016. Padrões de tipagem e autenticação em exames práticos de programação. In Anais da Conferência ACM de 2016 sobre Inovação e Tecnologia no Ensino de Ciência da Computação. 160–165.

[47] Juho Leinonen, Nea Pirttinen e Arto Hellas. 2020. Criação de conteúdo colaborativo para prática de SQL. Em Anais da Conferência ACM de 2020 sobre Inovação e Tecnologia no Ensino de Ciência da Computação. 349–355.

[48] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustín Dal Lago, et al. 2022. Geração de código de nível competitivo com AlphaCode. arXiv preprint arXiv:2203.07814 (2022).

[49] Raymond Lister. 2020. Sobre o desenvolvimento cognitivo do programador novato: e o desenvolvimento de um pesquisador em educação computacional. In Anais da 9ª Conferência de Pesquisa em Educação em Ciência da Computação (Evento Virtual, Holanda) (CSERC '20). Association for Computing Machinery, Nova Iorque, NY, EUA, Artigo 2, 15 páginas. <https://doi.org/10.1145/3442481.3442498>

[50] Raymond Lister, Elizabeth S. Adams, Sue Fitzgerald, William Fone, John Hamer, Morten Lindholm, Robert McCartney, Jan Erik Moström, Kate Sanders, Otto Seppälä, Beth Simon e Lynda Thomas. 2004. Um estudo multinacional das habilidades de leitura e rastreamento em programadores iniciantes. Em Relatórios do Grupo de Trabalho do ITICSE sobre Inovação e Tecnologia no Ensino de Ciência da Computação (Leeds, Reino Unido) (ITICSE-WGR '04). Association for Computing Machinery, Nova York, NY, EUA, 119–150. <https://doi.org/10.1145/1044550.1041673> [51] Raymond Lister, Colin Fidge e Donna Teague. 2009. Mais evidências de uma relação entre habilidades de explicação, rastreamento e escrita na programação introdutória. SIGCSE Bull. 41, 3 (jul. 2009), 161–165. <https://doi.org/10.1145/1595496.1562930>

[52] Raymond Lister, Beth Simon, Errol Thompson, Jacqueline L. Whalley e Christine Prasad. 2006. Não enxergando a floresta por causa das árvores: programadores novatos e a taxonomia SOLO. SIGCSE Bull. 38, 3 (jun. 2006), 118–122. <https://doi.org/10.1145/1140123.1140157>

[53] Richard Lobb e Jenny Harlow. 2016. Coderunner: Uma ferramenta para avaliar habilidades de programação de computadores. ACM Inroads 7, 1 (fev. 2016), 47–51. <https://doi.org/10.1145/2810041>

[54] Krista Longi, Juho Leinonen, Henrik Nygren, Joni Salmi, Arto Klami e Arto Vihavainen. 2015. Identificação de programadores a partir de padrões de tipagem. Em Anais da 15ª Conferência Koli Calling sobre pesquisa em educação computacional. 60–67.

[55] Renée McCauley, Sue Fitzgerald, Gary Lewandowski, Laurie Murphy, Beth Simon, Lynda Thomas e Carol Zander. 2008. Depuração: uma revisão

da literatura sob uma perspectiva educacional. *Computer Science Education* 18, 2 (2008), 67–92. <https://doi.org/10.1080/08993400802114581> arXiv:<https://doi.org/10.1080/08993400802114581>

[56] Laurie Murphy, Sue Fitzgerald, Raymond Lister e Renée McCauley. 2012. A capacidade de 'explicar em linguagem simples' está ligada à proficiência em programação baseada em computador. In *Anais da Nona Conferência Internacional Anual sobre Pesquisa em Educação em Computação* (Auckland, Nova Zelândia) (ICER '12). Association for Computing Machinery, Nova Iorque, NY, EUA, 111–118. <https://doi.org/10.1145/2361276.2361299> [57] Terence Nip, Elsa L. Gunter, Geoffrey L. Herman, Jason W. Morpew e Matthew West. 2018. Utilizando uma Instalação de Testes Baseada em Computador para Melhorar o Aprendizado do Aluno em um Curso de Linguagens de Programação e Compiladores. Em *Anais do 49º Simpósio Técnico da ACM sobre Educação em Ciência da Computação* (Baltimore, Maryland, EUA) (SIGCSE '18). Association for Computing Machinery, Nova York, NY, EUA, 568–573. <https://doi.org/10.1145/3159450.3159500> [58] Henrik Nygren, Juho Leinonen e Arto Hellas. 2019. Acesso não restrito a soluções de modelos: uma boa ideia? Em *Anais da Conferência ACM de 2019 sobre Inovação e Tecnologia no Ensino de Ciência da Computação*. 44–50.

[59] Henrik Nygren, Juho Leinonen, Nea Pirttinen, Antti Leinonen e Arto Hellas. 2019. Experimentando com soluções de modelos como mecanismo de suporte. In *Anais da 1ª Conferência de Pesquisa em Educação Computacional do Reino Unido e Irlanda*. 1–7.

[60] José Carlos Paiva, José Paulo Leal e Álvaro Figueira. 2022. Avaliação automatizada no ensino de ciência da computação: uma revisão do estado da arte. *ACM Transactions on Computing Education* (TOCE) (2022).

[61] Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri e Brendan Dolan-Gavitt. 2021. O OpenAI Codex e outros grandes modelos de linguagem podem nos ajudar a corrigir bugs de segurança? arXiv preprint arXiv:2112.02125 (2021).

[62] Hammond Pearce, Benjamin Tan, Prashanth Krishnamurthy, Farshad Khorrami, Ramesh Karri e Brendan Dolan-Gavitt. 2022. Teste rápido! Um modelo de linguagem grande pode ajudar na engenharia reversa? arXiv preprint arXiv:2202.01142 (2022).

[63] DN Perkins, Chris Hancock, Renee Hobbs, Fay Martin e Rebecca Simmons. 1986. Condições de Aprendizagem em Programadores Iniciantes. *Journal of Educational Computing Research* 2, 1 (1986), 37–55. <https://doi.org/10.2190/GUJT-JCBJ-Q6QU-Q9PL>

[64] Robert Phillips, Dan Lockton, Sharon Bauley e Sarah Silve. 2013. Fazendo instruções para outros: explorando modelos mentais por meio de um exercício simples. *Interactions* 20, 5 (set. 2013), 74–79. <https://doi.org/10.1145/2505290> [65] Nea Pirttinen, Vilma Kangas, Irene Nikkarinen, Henrik Nygren, Juho Leinonen e Arto Hellas. 2018. Crowdsourcing programming assignments with Crowd- Sorcerer. In *Proceedings of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education*. 326–331.

[66] Nea Pirttinen e Juho Leinonen. 2022. Os alunos podem avaliar seus colegas? Comparação entre avaliações de pares e de instrutores. In *Anais da 27ª Conferência da ACM sobre Inovação e Tecnologia no Ensino de Ciência da Computação*, Vol. 1.

[67] Leo Porter, Daniel Zingaro, Cynthia Lee, Cynthia Taylor, Kevin C. Webb e Michael Clancy. 2018. Desenvolvimento de objetivos de aprendizagem em nível de curso para estruturas de dados básicas em CS2. Em *Anais do 49º Simpósio Técnico da ACM sobre Educação em Ciência da Computação* (Baltimore, Maryland, EUA) (SIGCSE '18). Association for Computing Machinery, Nova York, NY, EUA, 858–863. <https://doi.org/10.1145/3159450.3159457> [68] Ruixiang Qi e Davide Fossati. 2020. Unlimited Trace Tutor: Aprendendo rastreamento de código com programas gerados automaticamente. Association for Computing Machinery , Nova York, NY, EUA, 427–433. <https://doi.org/10.1145/3328778.3366939> [69] Emily Q Rosenzweig, Allan Wigfield e Jacqueline S Eccles. 2019. Teoria da expectativa-valor e sua relevância para a motivação e aprendizagem do aluno. (2019).

[70] Sam Saarinen, Shriram Krishnamurthi, Kathi Fisler e Preston Tunnell Wilson. 2019. Aproveitando a sabedoria das turmas: terceirização de aulas e aprendizado de máquina para geração de instrumentos de avaliação. In *Anais do 50º Simpósio Técnico da ACM sobre Educação em Ciência da Computação* (Minneapolis, MN, EUA) (SIGCSE '19). Association for Computing Machinery, Nova Iorque, NY, EUA, 606–612. <https://doi.org/10.1145/3287324.3287504> [71] Kate Sanders, Marzieh Ahmadzadeh, Tony Clear, Stephen H Edwards, Mikey Goldweber, Chris Johnson, Raymond Lister, Robert McCartney, Elizabeth Patitsas e Jaime Spacco. 2013. O Canterbury QuestionBank: Construindo um repositório de questões de múltipla escolha de CS1 e CS2. Em *Anais da conferência de relatórios do grupo de trabalho ITICSE sobre Inovação e tecnologia no ensino de ciência da computação - relatórios do grupo de trabalho*. 33–52.

[72] Kate Sanders, Jonas Bousteid, Anna Eckerdal, Robert McCartney e Carol Zander . 2017. Pedagogia Popular: Ninguém Não Gosta de Aprendizagem Ativa. In *Anais da Conferência ACM de 2017 sobre Pesquisa Internacional em Educação Computacional* (Tacoma, Washington, EUA) (ICER '17). Association for Computing Machinery, Nova York, NY, EUA, 145–154. <https://doi.org/10.1145/3105726.3106192> [73] Patrick Schramowski, Cigdem Turan, Nico Andersen, Constantin A Rothkopf e Kristian Kersting. 2022. Grandes modelos de linguagem pré-treinados contêm vieses semelhantes aos humanos sobre o que é certo e errado fazer. *Nature Machine Intelligence* 4, 3 (2022), 258–268.

[74] Otto Seppälä, Petri Ihantola, Essi Isohanni, Juha Sorva e Arto Vihavainen. 2015. Sabemos o quão difícil é o problema da precipitação? In *Anais da 15ª Conferência Koli Calling sobre Pesquisa em Educação Computacional*. 87–96.

[75] Judy Sheard, Angela Carbone, Raymond Lister, Beth Simon, Errol Thompson e Jacqueline L. Whalley. 2008. Going SOLO to Assess Novice Programmers. In *Proceedings of the 13th Annual Conference on Innovation and Technology in Computer Science Education* (Madrid, Spain) (ITICSE '08). Association for Computing Machinery , New York, NY, USA, 209–213. <https://doi.org/10.1145/1384271.1384328> [76] Lee S Shulman. 2005. Pedagogias características nas profissões. *Daedalus* 134, 3 (2005), 52–59.

[77] Valerie J Shute. 2008. Foco no feedback formativo. *Revisão de pesquisa educacional* 78, 1 (2008), 153–189.

[78] Anjali Singh, Christopher Brooks, Yiwen Lin e Warren Li. 2021. O que os alunos ganham com isso? Evidências de um experimento de campo randomizado sobre perguntas de pesquisa de alunos em um MOOC. Em *Anais da Oitava Conferência ACM sobre Aprendizagem em Escala*. 221–233.

[79] E. Soloway. 1986. Aprender a programar = Aprender a construir mecanismos e explicações. *Commun. ACM* 29, 9 (set. 1986), 850–858. <https://doi.org/10.1145/6592.6594>

[80] Elliot Soloway e Kate Ehrlich. 1984. Estudos empíricos de conhecimento de programação . *IEEE Transactions on software engineering* 5 (1984), 595–609.

[81] Ben Stephenson. 2018. Uma experiência com o uso de questões de programação no computador durante exames. In *Anais da 23ª Conferência Canadense Ocidental sobre Educação em Computação* (Victoria, BC, Canadá) (WCCCE '18). Association for Computing Machinery, Nova York, NY, EUA, Artigo 11, 6 páginas. <https://doi.org/10.1145/3209635.3209639>

[82] Zahid Ullah, Adidah Lajis, Mona Jamjoom, Abdulrahman Altalhi, Abdullah Al- Ghamdi e Farrukh Saleem. 2018. O efeito da avaliação automática na programação de iniciantes: pontos fortes e limitações dos sistemas existentes. *Computer Applications in Engineering Education* 26, 6 (2018), 2328–2341. <https://doi.org/10.1002/cae.21974> arXiv:<https://doi.org/10.1002/cae.21974> [83] Anne Venables, Grace Tan e Raymond Lister. 2009. Uma análise mais detalhada das habilidades de rastreamento, explicação e escrita de código no programador iniciante. Em *Anais do Quinto Workshop Internacional sobre Pesquisa em Educação Computacional* (Berkeley, CA, EUA) (ICER '09). Association for Computing Machinery, Nova York, NY, EUA, 117–128. <https://doi.org/10.1145/1584322.1584336> [84] Arto Vihavainen, Jonne Airaksinen e Christopher Watson. 2014. Uma revisão sistemática de abordagens para o ensino de programação introdutória e sua influência no sucesso. Em *Anais da décima conferência anual sobre pesquisa internacional em educação em computação*. 19–26.

[85] Arto Vihavainen, Matti Paksula e Matti Luukkainen. 2011. Método de aprendizagem extremo no ensino de programação para iniciantes. Nos *Anais do 42º Simpósio Técnico ACM sobre Ensino de Ciência da Computação*. 93–98.

[86] Regina Vollmeyer e Falko Rheinberg. 2005. Um efeito surpreendente do feedback sobre aprendizagem. *Aprendizagem e instrução* 15, 6 (2005), 589–602.

[87] Lev Semenovich Vygotsky e Michael Cole. 1978. A mente na sociedade: Desenvolvimento dos processos psicológicos superiores. Harvard University Press.

[88] Jacqueline L. Whalley, Raymond Lister, Errol Thompson, Tony Clear, Phil Robbins, PK Ajith Kumar e Christine Prasad. 2006. Um estudo australiano sobre habilidades de leitura e compreensão em programadores iniciantes, usando as taxonomias de Bloom e SOLO. Em *Anais da 8ª Conferência Australiana sobre Educação em Computação - Volume 52* (Hobart, Austrália) (ACE '06). Sociedade Australiana de Computação, Inc., AUS, 243–252.

[89] Joseph Jay Williams, Juho Kim, Anna Rafferty, Samuel Maldonado, Krzysztof Z. Gajos, Walter S. Lasecki e Neil Heffernan. 2016. AXIS: Gerando explicações em escala com Learnersourcing e aprendizado de máquina. In *Anais da Terceira Conferência ACM sobre Aprendizado em Escala* (Edimburgo, Escócia, Reino Unido) (L@S '16). Association for Computing Machinery, Nova Iorque, NY, EUA, 379–388. <https://doi.org/10.1145/2876034.2876042> [90] Laurie Williams, Robert R Kessler, Ward Cunningham e Ron Jeffries. 2000. Fortalecendo o argumento a favor da programação em pares. *IEEE software* 17, 4 (2000), 19–25.

[91] John Wrenn, Shriram Krishnamurthi e Kathi Fisler. 2018. Quem testa os testadores?. In *Anais da Conferência ACM de 2018 sobre Pesquisa Internacional em Educação em Computação* (Espoo, Finlândia) (ICER '18). Association for Computing Machinery, Nova York, NY, EUA, 51–59. <https://doi.org/10.1145/3230977.3230999> [92] Benjamin Xie, Dastyni Loksa, Greg L. Nelson, Matthew J. Davidson, Dongsheng Dong, Harrison Kwik, Alex Hui Tan, Leanne Hwa, Min Li e Amy J. Ko. 2019. Uma Teoria de Instrução para Habilidades Introdutórias de Programação. *Educação em Ciência da Computação* 29, 2-3 (2019), 205–253. <https://doi.org/10.1080/08993408.2019.1565235> [93] Benjamin Xie, Greg L. Nelson e Amy J. Ko. 2018. Uma estratégia explícita para auxiliar o rastreamento de programas por iniciantes. Em *Anais do 49º Simpósio Técnico da ACM sobre Educação em Ciência da Computação* (Baltimore, Maryland, EUA) (SIGCSE '18). Association for Computing Machinery, Nova Iorque, NY, EUA, 344–349. <https://doi.org/10.1145/3159450.3159527>

[94] Lisa Yan, Annie Hu e Chris Piech. 2019. Pensieve: Feedback on Coding Process for Novices. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education* (Minneapolis, MN, USA) (SIGCSE '19). Association for Computing Machinery, New York, NY, USA, 253–259.

Geração automática de exercícios de programação e explicações de código usando grandes modelos de linguagem. ICER 2022, 7 a 11 de agosto de 2022, Lugano e evento virtual, Suíça.

EXEMPLOS DE SOLUÇÕES PARA PROGRAMAÇÃO

EXERCÍCIOS DESCRITOS EM 3.2.1

```
def speed_check (velocidade):
    se a velocidade for maior que 120:
        retornar "Você foi multado em US$ 200"
    elif speed > 100:
        returnelse : "Você foi multado em US$ 100"

    retornar " Tudo certo , corrida à frente"

imprimir (velocidade _ verificar (88))
imprimir (velocidade _ verificar (110))
imprimir (velocidade _ verificar (130))
```

```
def efervescente _ zumbido (números):
    para número em números:
        Se número % 3 == 0 e número % 5 == 0:
            imprimir ( " Fizz Buzz " )
        elif number % 3 == 0:
            imprimir ( " Fizz " )
        senão se número % 5 == 0:
            imprimir ( " Buzz " )
    outro:
        imprimir (número)
```

```
total = 0
contagem = 0

enquanto Verdadeiro:

    valor = int ( entrada ( " Escreva o valor se valor == 9 , 9999 fins. " ) )
    999:
        quebrar

    se valor < 0 ou valor > 1000:
        imprimir ( " Entrada inválida " )
        continuar

    total += valor
    contagem += 1

se a contagem for igual a 0:
    imprimir ( " Nenhuma entrada " )
outro:
    imprimir ( f " Ave raiva : { total / contagem } " )
```

```
Conversor classe C():
    def init __ ( selfrates ):
        , intercâmbio _
        troca própria _ taxas = câmbio _ taxas

    def converter ( self y , de om_currency, para _ moeda ,
        quantidade ) :
        valor_em_usd = valor / self.exchange y from _ taxas [
            _ moeda ]
        retornar amoun t_in_usdcurrency ] y troca própria _ taxas [
            y para _

conversor = Conversor ( { "USD" : y : 0.75 } ) 1 , "EUR" : 0.9 , "GBP"

imprimir ( conversor . converter ( "EUR" , "GBP" , 100 ) )
```


Geração automática de exercícios de programação e explicações de código usando grandes modelos de linguagem. ICER 2022, 7 a 11 de agosto de 2022, Lugano e evento virtual, Suíça.

Exemplo completo de priming com C

TEMAÇÃO DE TEMAS PARA PROGRAMAÇÃO
GERAÇÃO DE EXERCÍCIOS

Exemplo de priming (entrada)

```
""" Exercício 1
Palavras-chave
carros
função
para am eters
condicional
Enunciado do problema
Escreva uma função chamada speed di ng _ checkthattakes
    y asinglepar am eterspeed e printsout y Você é
multado em 200 dólares se a velocidade for superior a 1 2 0 , " Você
    y refinado por US$ 100 " y 100 mas se a velocidade for superior
abaixo de 120 e outras impressões y adiante " " Tudo certo , corrida
Solução de amostra
defspee di ng _ check (velocidade):
    ifspeed > 1 2 0 :
        retornar "Você foi multado em US$ 200"
    elifspeed > 1 0 0 :
        retornar "Você foi multado em US$ 100"
    outro:
        retornar " Tudo certo , corrida à frente"
Testes
classe Teste ( unittest . Teste Caso ) :
    mais habilitado _ verificação de velocidade (auto) :
        self.assertE quals(speed _ check(100), ybom, 'corrida à frente')
        self.assertE quals(speeding _ check(101), yarefinedfor$100)
        self.assertE quals(speed _ check(121), yarefinedfor$200)
""" Exercício 2
Palavras-chave
hóquei no gelo
Enunciado do problema
```

Exercício gerado (saída com temperatura 0)

```
Escreva uma função chamada gelo _ hóquei _ confira isso
    y takesasinglepar am eter
O placar é exibido e impresso y penalidade por Você recebeu um prêmio
    conduta antidesportiva se a pontuação for maior que 5 e caso contrário
    y impressões "Tudo bem, continue jogando."
Solução de amostra
def _ hóquei _ verificar ( pontuação ) :
    ifscore > 5:
        retornar "Você recebeu uma penalidade por
    y antidesportivo como keconductelse :
        retornar "Tudo bem, continue jogando."
Testes
classe Teste ( unittest . T e s t C a s e ) :
    mais habilitado _ gelo _ hóquei _ checar ( si mesmo ) :
        self.assertEqual(ice y bom, continue _ hóquei _ verificar ( 5 ) , ' Um todo
        jogando)
        _ hóquei _ check ( 6 ) . y são ' Você
        concedidos uma penalidade por conduta antidesportiva como ke
        y c on duct ' )
```